

Rangkuman Materi Perkuliahan Machine Learning Kelompok 22

Muhammad Ichsan Junaedi¹, Amanda Wijayanti²

^{1,2} Sistem Informasi, Sekolah Tinggi Manajemen Informatika Komputer (STMIK) Tazkia

E-mail :¹ 2415720100024.ichsan@student.stmik.tazkia.ac.id

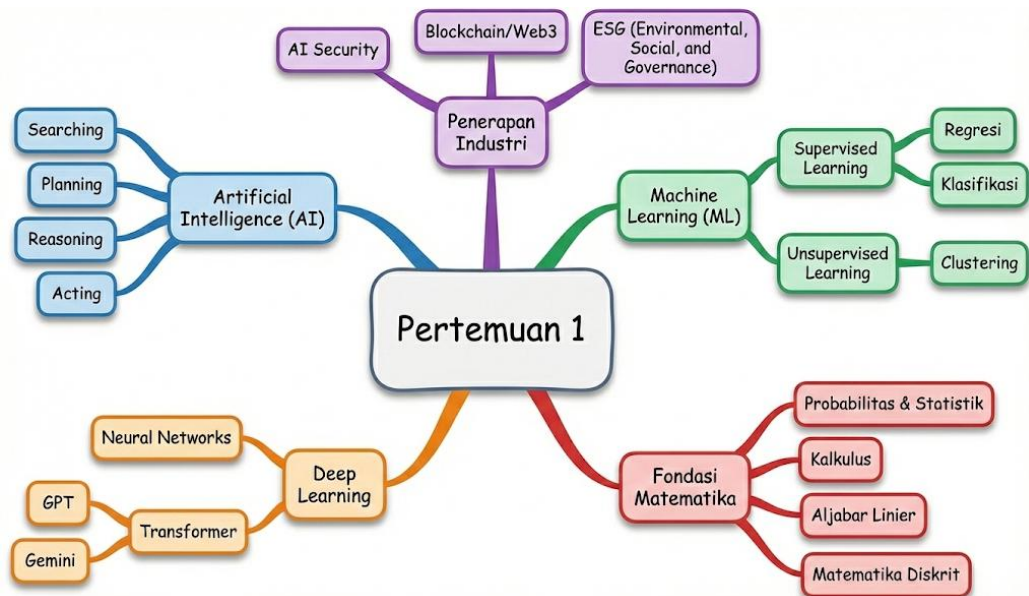
² 241572010006.amanda@student.stmik.tazkia.ac.id

BAB 1 Pengantar Machine Learning

1. Deskripsi Singkat Pembahasan

Pertemuan ini membahas landasan fundamental Machine Learning (ML) sebagai bagian integral dari Artificial Intelligence (AI). Fokus utama adalah memahami kapan sebuah komputer dapat dikatakan "belajar" melalui pendekatan formal yang dikemukakan oleh Tom Mitchell. Selain itu, dibahas pula kaitan erat antara ML dengan statistik (Propstat), matematika linier, serta peran strategis data sebagai "bahan bakar" pengetahuan. Bahasan juga meluas ke ranah profesional, mengidentifikasi peran-peran kunci seperti Data Scientist dan AI Engineer, serta tantangan masa depan terkait keamanan AI dan kepatuhan (compliance) data.

2. Mindmap



3. Penjelasan Detail

3.1 Definisi Machine Learning (Perspektif Tom Mitchell)

Berdasarkan literatur klasik *Introduction to Machine Learning* oleh Tom Mitchell, sebuah program komputer dikatakan belajar dari pengalaman E sehubungan dengan beberapa kelas tugas T dan ukuran kinerja P , jika kinerjanya pada tugas-tugas di T , sebagaimana diukur oleh P , meningkat dengan pengalaman E .

- Task (T): Tugas spesifik yang dikerjakan (misal: Klasifikasi email spam, ujian aljabar).
- Performance (P): Metrik evaluasi untuk mengukur keberhasilan (misal: Akurasi, nilai ujian).
- Experience (E): Data atau interaksi masa lalu yang digunakan untuk belajar (misal: Kumpulan email lama, proses belajar satu semester).

3.2 Hubungan Data, Fakta, dan Pengetahuan

Data didefinisikan sebagai Fakta. Dalam ML, pengalaman (E) adalah sekumpulan data/fakta yang sudah terjadi. Proses pembelajaran bertujuan menghasilkan Pengetahuan (Knowledge) atau Model/Pattern. Pengetahuan yang dihasilkan harus konsisten terhadap data. Jika terjadi perubahan pada fakta di lapangan, maka model atau hipotesis harus diperbarui agar tetap relevan.

3.3 Strategi Regularisasi dan Optimasi

Industri saat ini sangat membutuhkan tenaga ahli yang memahami fundamental, bukan sekadar praktisi bootcamp. Terdapat beberapa peran utama:

- Data Scientist: Fokus pada penggalian wawasan dari data.
- AI/ML Engineer: Fokus pada implementasi model ke dalam perangkat lunak.
- Data Analyst: Fokus pada visualisasi dan interpretasi data (memiliki entry barrier paling rendah).
- Solution Architect: Merancang arsitektur solusi AI secara menyeluruh.

3.4 Tantangan AI Security dan Compliance

Di masa depan (5 tahun ke depan), bidang AI Security dan Compliance (seperti UU PDP di Indonesia atau GDPR di Eropa) akan menjadi sangat krusial. Perusahaan tidak hanya butuh AI yang pintar, tapi AI yang aman dan mematuhi aturan privasi data pribadi.

4. Sample Code (Pseudocode)

```
# Pseudocode Proses Belajar Sederhana (Gradient Descent Logic)
initialize model_weights
data = get_experience_E()
target_task = T

for epoch in range(iterations):
    prediction = model_weights * data # Melakukan Task T
    performance_score = evaluate(prediction, target_task) # Mengukur P

    error = target_task - prediction

    if error != 0:
        # Belajar dari pengalaman (E) untuk memperbaiki model
        model_weights = model_weights + (learning_rate * error * data)
        print(f"Update weights. New Performance Score: {performance_score}")
```

5. Kesimpulan

Transisi definisi dari kecerdasan buatan umum menuju mekanisme pembelajaran komputer yang terukur. Menggunakan rujukan formal dari **Tom Mitchell**, ditegaskan bahwa komputer dikatakan belajar jika mampu meningkatkan kinerjanya (P) pada suatu tugas (T) berdasarkan pengalaman (E).

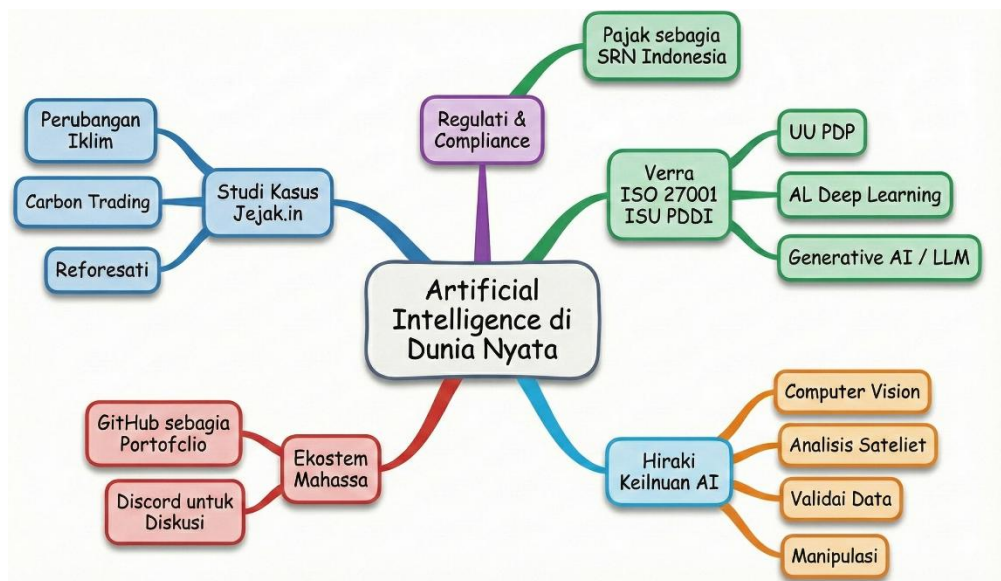
ML diposisikan bukan sekadar alat *coding*, melainkan sistem yang memproses fakta (data) menjadi pengetahuan yang konsisten. Selain itu, bab ini mengidentifikasi peran profesional seperti *Data Scientist* dan *AI Engineer* serta menyoroti tren masa depan seperti **ESG** (*Environmental, Social, and Governance*) dan **AI Security**. Mahasiswa ditekankan untuk memiliki pemahaman fundamental (matematika) agar mampu beradaptasi dengan teknologi yang terus berevolusi.

BAB 2 Implementasi AI di Dunia Nyata dan Peta Besar Artificial Intelligence

1. Deskripsi Singkat Pembahasan

Pertemuan kedua ini memfokuskan pada peran strategis *Machine Learning* dalam mendukung keberlanjutan lingkungan melalui proyek karbon dan *ESG (Environmental, Social, and Governance)*. Pembahasan mencakup bagaimana AI digunakan sebagai alat validasi otomatis dalam reforestasi, penggunaan *blockchain* untuk sertifikasi karbon, serta pendalaman taksonomi AI yang membedakan antara sistem pakar (*Expert System*) dan *Machine Learning* berdasarkan alur logikanya (Induktif vs Deduktif).

2. Mindmap



3. Penjelasan Detail Materi

3.1 Studi Kasus: AI dalam Pasar Karbon (Carbon Trading)

Dunia saat ini menghadapi ancaman kenaikan permukaan laut (seperti ancaman tenggelamnya Jakarta). Perusahaan global melakukan mitigasi dengan membeli sertifikat karbon melalui proyek reforestasi. AI berperan krusial dalam proses **Automation & Validation**:

- **Computer Vision:** Mengidentifikasi jenis pohon melalui foto (misal: membedakan bibit durian dan mangga) untuk validasi klaim karbon.
- **Satellite Imagery:** Memverifikasi tutupan lahan dan status legal tanah (hutan lindung vs lahan pribadi) guna memastikan kepatuhan (*compliance*).

3.2 Compliance dan Regulasi

Implementasi IT tidak lepas dari aturan hukum (*Compliance*). Di bidang keamanan dan lingkungan, terdapat standar yang harus dipatuhi:

- **Lokal:** UU PDP (Perlindungan Data Pribadi), SRN (Sistem Registrasi Nasional), IDX Carbon.
- **Internasional:** ISO 27001 (Security), GDPR, Vera (Standar Karbon Global), GRI.

3.3 Induksi vs Deduksi dalam AI

Perbedaan fundamental antara AI lama dan ML modern terletak pada cara membangun pengetahuan:

1. **Sistem Pakar (Deduktif):** Mengandalkan input aturan dari manusia (*If-Then*). Jika aturan tidak lengkap, sistem tidak bisa berkembang.
2. **Machine Learning (Induktif):** Mesin diberikan data (pengalaman), lalu mesin secara mandiri mencari aturan atau pola dari data tersebut. Inilah inti dari *Machine Learning*.

4. Sample Code (Logic Comparison)

Perbandingan antara pendekatan Rule-Based (Expert System) dan Machine Learning sederhana:

Pendekatan Deduktif (Expert System/Rule-Based)

```
def check_forest_claim(tree_type, land_status):
    if tree_type == "Hutan" and land_status == "Pribadi":
        return "Boleh Klaim Karbon"
    else:
        return "Tidak Boleh Klaim"
```

Pendekatan Induktif (Machine Learning Pattern)

Mesin belajar pola: [(Hutan, Pribadi) -> Klaim]

Jika ada data baru (Rawa, Pribadi), mesin mencoba memprediksi nilainya.

1. Kesimpulan

Pada bab ini, fokus beralih pada penerapan nyata AI dalam mendukung keberlanjutan lingkungan melalui Carbon Trading. Kesimpulan pentingnya adalah peran AI sebagai validator otomatis dalam reforestasi—seperti membedakan jenis flora melalui citra medis atau memverifikasi tutupan lahan melalui satelit. Integrasi Blockchain juga diperkenalkan sebagai solusi untuk menjamin keamanan sertifikat karbon agar tidak diklaim ganda.

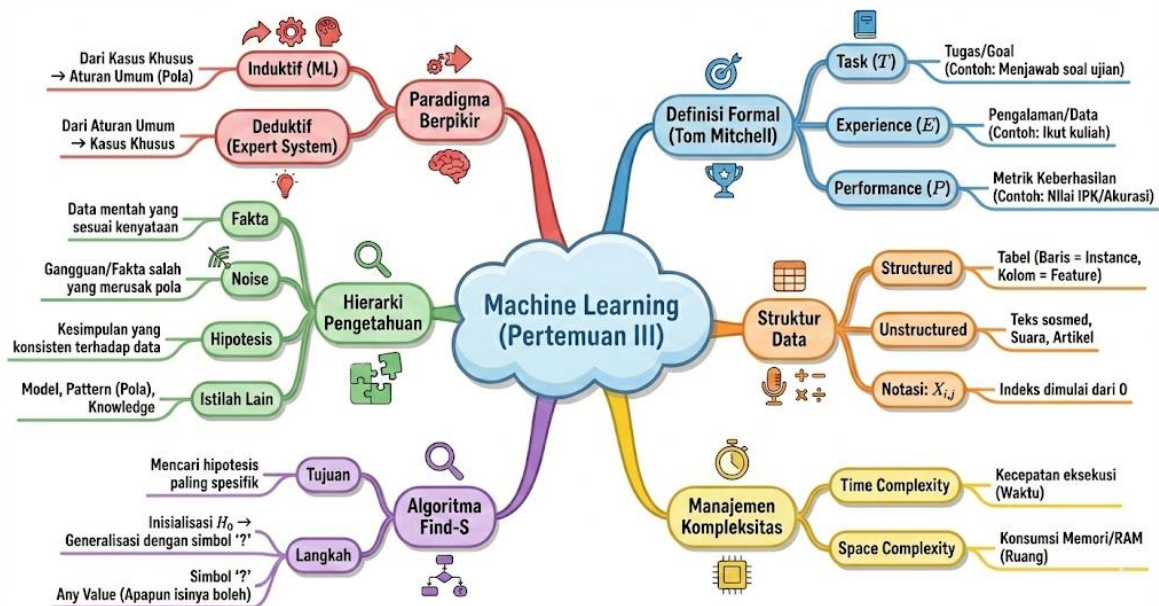
Secara teoretis, bab ini membedah perbedaan mendalam antara Sistem Pakar (Deduktif) yang bergerak dari aturan umum ke khusus, dengan Machine Learning (Induktif) yang bergerak dari data spesifik menuju pola umum. Terakhir, ditekankan aspek Compliance (Kepatuhan); di mana seorang *engineer* AI harus menaati regulasi hukum seperti UU PDP di Indonesia atau GDPR di Eropa agar model yang dibangun tidak hanya cerdas, tetapi juga legal dan etis

BAB 3 Konsep Pembelajaran Induktif dan Implementasi Algoritma Find-S pada Machine Learning

1. Deskripsi Singkat Bahasan

Pertemuan ketiga ini mendalami konsep fundamental pembelajaran mesin sebagai proses Induksi. Bahasan dimulai dari pemantapan definisi formal pembelajaran menurut Tom Mitchell, pemetaan data terstruktur dan tidak terstruktur, hingga demonstrasi teknis algoritma Find-S (Find-Specific). Fokus utama adalah memahami bagaimana komputer melakukan generalisasi dari data spesifik (fakta) menuju pembentukan hipotesis yang konsisten.

2. Mindmap



3. Penjelasan Detail

3.1 Definisi Formal dan Analogi Pembelajaran

Berdasarkan literatur *Introduction to Machine Learning*, komputer dikatakan belajar melalui tiga variabel utama:

- **Task (T):** Tugas yang dikerjakan (Contoh: Menjawab soal ujian).
- **Performance Measure (P):** Ukuran keberhasilan (Contoh: Nilai akumulatif tugas dan ujian).
- **Experience (E):** Data atau aktivitas yang memberikan wawasan (Contoh: Mengikuti perkuliahan selama satu semester).

Sistem dikatakan belajar apabila nilai P meningkat seiring bertambahnya E . Jika mahasiswa (sebagai analogi mesin) mengulang kuliah karena nilai di bawah 70, proses pengulangan tersebut adalah upaya menambah E untuk memperbaiki P pada tugas T .

3.2 Taksonomi Data dan Pengetahuan

Data dipahami sebagai **Fakta Objektif**. Dalam *Machine Learning*, data terbagi menjadi dua kategori:

1. **Structured Data:** Data yang tersusun rapi dalam tabel (baris sebagai *instance* dan kolom sebagai *field/feature*).
2. **Unstructured Data:** Data acak seperti teks media sosial, suara, atau artikel berita yang harus dikonversi menjadi *structured* agar bisa diproses oleh algoritma.

Pengetahuan (*Knowledge*) dalam ML sering kali disebut sebagai **Hipotesis (h)**. Hipotesis adalah kesimpulan yang harus konsisten terhadap data. Jika terjadi perubahan fakta di lapangan (seperti perubahan cuaca ekstrem), maka hipotesis tersebut harus berubah mengikuti data terbaru.

3.4 Algoritma Find-S (Specific-to-General)

Algoritma Find-S digunakan untuk mencari hipotesis paling spesifik yang mampu mencakup seluruh contoh positif dalam sebuah dataset.

- **Himpunan Hipotesis:** Dimulai dari H_0 yang sangat spesifik (simbol $\emptyset$ atau tidak menerima nilai apa pun).
- **Generalisasi:** Jika sistem menemukan data positif, sistem akan membandingkan atribut data dengan hipotesis. Jika berbeda, atribut tersebut diganti dengan simbol "?" (*General/Any Value*).
- **Karakteristik:** Algoritma ini hanya belajar dari contoh positif dan mengabaikan contoh negatif untuk memastikan model tidak menjadi terlalu luas (*underfitting*).

3.5 Manajemen Kompleksitas Komputasi

Pengembangan model ML harus mempertimbangkan dua jenis kompleksitas:

1. **Time Complexity:** Kecepatan mesin dalam memproses data dan memberikan prediksi.
2. **Space Complexity:** Kapasitas memori (RAM) yang dikonsumsi model. Fitur yang ditandai dengan "?" dianggap tidak signifikan sehingga sering kali diabaikan untuk mengoptimalkan ruang memori.

4. Sample code & Pseudocode

```
import numpy as np

# 1. Definisi Dataset Terstruktur (Structured Data)
# Baris = Instance, Kolom = Feature/Atribut
# Indeks matriks dimulai dari 0 sesuai standar komputasi
data_fakta = np.array([
    ['Cerah', 'Panas', 'Normal', 'Kuat'], # x_0 (Positif)
    ['Cerah', 'Panas', 'Tinggi', 'Kuat'], # x_1 (Positif)
    ['Hujan', 'Dingin', 'Tinggi', 'Kuat'], # x_2 (Negatif)
    ['Cerah', 'Panas', 'Tinggi', 'Lemah'] # x_3 (Positif)
])

# Label Target (Y): Apakah "Bermain" (Yes/No)
target = np.array(['Yes', 'Yes', 'No', 'Yes'])

def find_s_algorithm(X, y):
    # 2. Inisialisasi Hipotesis paling spesifik (Specific Hypothesis)
    # Mengambil baris positif pertama sebagai h_0
    for i, label in enumerate(y):
        if label == 'Yes':
            h = X[i].copy()
            break

    print(f"Hipotesis Awal (h_0): {h}")
    print("-" * 30)

    # 3. Proses Induksi (Generalisasi)
    for i in range(len(X)):
        if y[i] == 'Yes':
            for j in range(len(h)):
                # Membandingkan fakta x_i dengan hipotesis h
                if X[i][j] != h[j]:
                    # Generalisasi: Ganti dengan simbol '?'
                    h[j] = '?'
            print(f"Update h setelah instance ke-{i}: {h}")
        else:
            # Mengabaikan contoh negatif (Manajemen Kompleksitas)
            print(f"Instance ke-{i} Negatif: Diabaikan.")

    return h
```

```
# Eksekusi Program
hipotesis_final = find_s_algorithm(data_fakta, target)

print("-" * 30)
print(f"Pengetahuan (Knowledge) yang terbentuk: {hipotesis_final}")
```

5. Kesimpulan

Pertemuan ketiga mendalami mekanisme teknis bagaimana mesin membentuk pengetahuan melalui proses induksi. Kesimpulannya adalah bahwa pengetahuan dalam ML (disebut sebagai hipotesis atau model) bersifat **stokastik** dan harus konsisten terhadap data (fakta). Data sendiri dikategorikan menjadi terstruktur (tabel) dan tidak terstruktur (teks/suara), dengan notasi matriks $X_{i,j}$ di mana indeks dimulai dari 0.

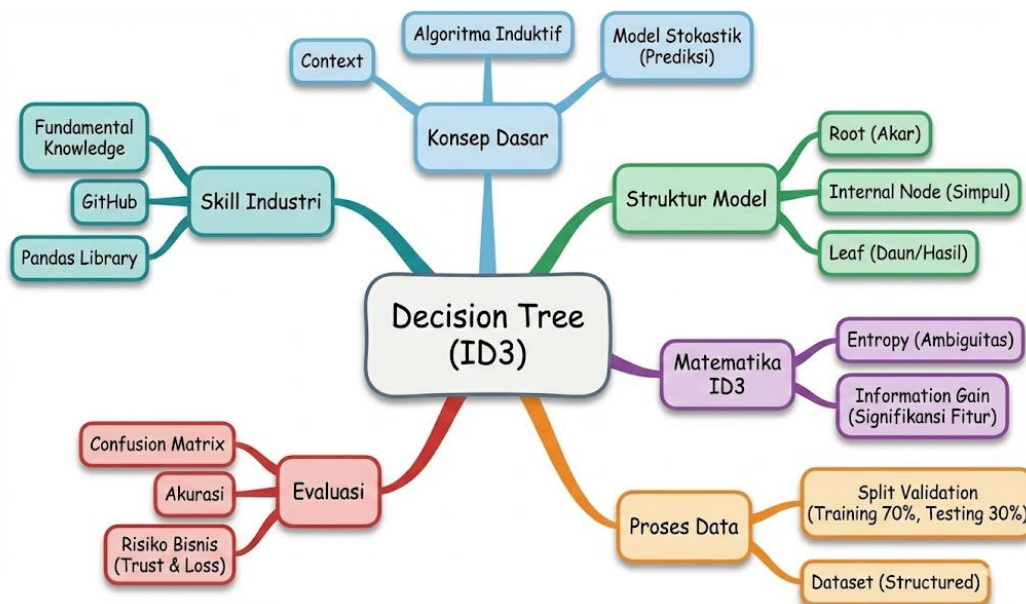
Bab ini memperkenalkan algoritma **Find-S** (*Find-Specific*) sebagai metode *Concept Learning* pertama. Algoritma ini bekerja dengan memulai dari hipotesis paling spesifik dan berangsur menjadi lebih umum melalui penggunaan simbol "?" (*any value*) setiap kali menemukan contoh positif yang berbeda. Selain itu, manajemen kompleksitas (**Time** dan **Space**) menjadi catatan penting; fitur yang tidak signifikan harus dieliminasi demi efisiensi memori sistem.

BAB 4 Algoritma Pohon Keputusan (Decision Tree) ID3 dan Evaluasi Model melalui Confusion Matrix

1. Deskripsi Singkat Pembahasan

Pertemuan keempat ini merupakan transisi penting dari metode *specific-to-general* (Find-S) menuju algoritma berbasis struktur pohon, khususnya **ID3 (Iterative Dichotomiser 3)**. Bahasan mencakup distingsi antara algoritma dan model, mekanisme pemilihan *root* melalui *Information Gain*, serta evaluasi model menggunakan metode *split validation* dan *confusion matrix*. Penekanan khusus diberikan pada dampak bisnis dari kesalahan prediksi model (seperti *underfitting* dan *overfitting*).

2. Mindmap



3. Penjelasan Detail

3.1 Taksonomi Algoritma dan Model

Penting untuk membedakan antara Metode/Algoritma (cara membentuk pengetahuan) dan Model (produk yang dihasilkan).

- Algoritma: ID3, C4.5, *Backpropagation*, atau *Convolutional Neural*.
- Model/Output: Berupa *Tree* (Pohon), *Matrix*, *Vektor*, atau *Graph* (seperti pada *Bayesian Networks*).

Model ML bersifat Stokastik (hasil prediksi/probabilitas), berbeda dengan sistem tradisional yang bersifat Deterministik (pasti). Inilah mengapa evaluasi akurasi menjadi tahapan wajib dalam siklus hidup pengembangan AI.

3.2 Mekanisme Kerja Algoritma ID3

Ide pokok dari ID3 adalah menentukan Signifikansi Fitur. Algoritma ini mencari fitur mana yang paling berkontribusi dalam membedakan hasil akhir (kelas/label). Proses ini dilakukan melalui dua perhitungan matematis utama:

- Entropy (H): Ukuran ketidakteraturan atau ambiguitas dalam sebuah dataset.

$$H(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

- Information Gain (*Gain*): Pengurangan entropi setelah data set dibagi berdasarkan atribut tertentu. Fitur dengan Gain tertinggi dipilih menjadi Root atau simpul keputusan.

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

3.3 Evaluasi Model dan Dampak Bisnis

Model yang dihasilkan tidak langsung diimplementasikan, melainkan diuji menggunakan metode Split Validation (umumnya rasio 70% *training* dan 30% *testing*). Hasil pengujian dipetakan ke dalam Confusion Matrix untuk melihat:

- True Positive/Negative: Prediksi yang benar.
- False Positive/Negative: Prediksi yang salah.

Kesalahan prediksi memiliki risiko bisnis yang nyata. Misalnya dalam sistem deteksi *fraud*:

- Underfit (Terlalu Longgar): Banyak transaksi penipuan yang lolos (merugikan secara finansial dan hukum).
- Overfit (Terlalu Ketat): Transaksi pelanggan jujur ditolak (merugikan *trust* pelanggan dan hilangnya potensi transaksi).

4. Sample Code & Pseudocode

Bagian ini menyajikan logika di balik pembentukan pohon keputusan menggunakan algoritma ID3 dan cara menghitung performa model tersebut secara statistik.

4.1 Pseudocode Algoritma ID3 (Recursive Induction)

Pseudocode ini menjelaskan bagaimana algoritma memilih atribut terbaik secara berulang untuk membangun simpul (*node*).

Algorithm ID3(Examples, Target_Attribute, Attributes)

1. Buat simpul Root untuk pohon.
2. Jika semua Examples bernilai positif, kembalikan simpul Root dengan label "+".
3. Jika semua Examples bernilai negatif, kembalikan simpul Root dengan label "-".
4. Jika Attributes kosong, kembalikan Root dengan label nilai target paling umum dalam Examples.
5. Selain itu:
 - $A \leftarrow$ Atribut dalam Attributes yang memiliki *Information Gain* tertinggi terhadap Examples.
 - Tetapkan A sebagai atribut keputusan untuk Root.
 - Untuk setiap nilai v_i dari A :
 - Tambahkan cabang baru di bawah Root untuk pengujian $A = v_i$.
 - $Examples(v_i) \leftarrow$ subset Examples yang memiliki nilai v_i untuk A .
 - Jika $Examples(v_i)$ kosong, tambahkan simpul daun dengan label nilai target paling umum dalam Examples.
 - Selain itu, tambahkan di bawah cabang baru tersebut subpohon: $ID3(Examples(v_i), Target_Attribute, Attributes - A)$.
6. Kembalikan Root.

4.2 Implementasi Perhitungan Entropy & Gain (Python)

Berikut adalah cuplikan kode Python untuk menghitung Entropy dan Information Gain, yang merupakan "jantung" dari algoritma ID3.

```
import math

def calculate_entropy(data_labels):
    """Menghitung Entropy H(S)"""
    total_count = len(data_labels)
    counts = {}
    for label in data_labels:
        counts[label] = counts.get(label, 0) + 1
```

```

entropy = 0
for label in counts:
    probability = counts[label] / total_count
    # Rumus LaTeX: H(S) = -sum(p_i * log2(p_i))
    entropy -= probability * math.log2(probability)
return entropy

def calculate_information_gain(total_entropy, subset_entropies, subset_sizes, total_size):
    """Menghitung Information Gain(S, A)"""
    weighted_entropy = 0
    for i in range(len(subset_entropies)):
        weighted_entropy += (subset_sizes[i] / total_size) * subset_entropies[i]

    # Gain = Entropy awal - weighted entropy dari atribut
    return total_entropy - weighted_entropy

# Contoh Penggunaan Sederhana
labels_awal = ['Yes', 'Yes', 'No', 'No', 'Yes'] # Dataset S
h_s = calculate_entropy(labels_awal)
print(f"Entropy H(S): {h_s:.4f}")

```

4.3 Evaluasi dengan Confusion Matrix

Setelah model pohon terbentuk, kita melakukan evaluasi menggunakan *testing data*. Hasilnya dipetakan ke dalam Confusion Matrix.

```

from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# Simulasi Hasil Prediksi vs Fakta (Testing Data)
y_true = [1, 0, 1, 1, 0, 1, 0, 1, 0] # Fakta (Ground Truth)
y_pred = [1, 0, 1, 0, 0, 1, 1, 0, 1] # Hasil Prediksi Model

# Membangun Matrix
cm = confusion_matrix(y_true, y_pred)

print("Confusion Matrix:")
print(cm)

# Penjelasan Dampak Bisnis (Overfit/Underfit)
# Baris ke-2, Kolom ke-1 (False Negative): Prediksi Fraud tapi dianggap Normal (Underfit)
# Baris ke-1, Kolom ke-2 (False Positive): Prediksi Normal tapi dianggap Fraud (Overfit)

```

5. Kesimpulan

Ini menyimpulkan transisi ke algoritma yang lebih aplikatif di industri, yaitu **Decision Tree** varian **ID3**. Perbedaan antara algoritma (proses pengolahan) dan model (produk akhir berupa pohon/graf) dijelaskan secara mendalam. Mekanisme utama ID3 adalah pencarian fitur paling signifikan melalui perhitungan **Entropy** (ambiguitas data) dan **Information Gain** (penurunan ambiguitas).

$$H(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

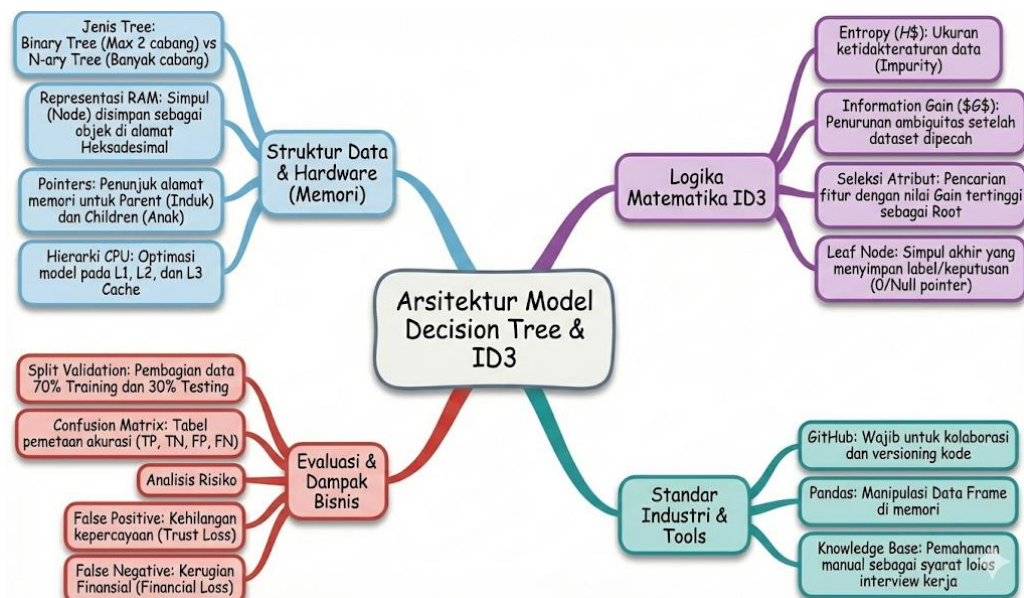
Aspek evaluasi menjadi poin krusial dalam bab ini. Pengujian model melalui **Split Validation** (70:30) dan pemetaan hasil pada **Confusion Matrix** diperlukan untuk mendeteksi risiko **Underfitting** (model terlalu longgar) dan **Overfitting** (model terlalu ketat). Kesalahan prediksi ini disimpulkan memiliki dampak bisnis yang nyata, seperti potensi kerugian finansial atau hilangnya kepercayaan pelanggan dalam sistem deteksi *fraud*.

BAB 5 Arsitektur Model Tree dan Kalkulasi ID3

1. Deskripsi Singkat Pembahasan

Pertemuan kelima ini merupakan pendalaman teknis dari algoritma *Decision Tree*. Fokus utama pembahasan beralih dari sekadar konsep teoretis menuju mekanisme Implementasi Struktur Data pada tingkat memori (*hardware-level*). Materi mencakup cara CPU dan RAM menyimpan simpul (*node*) menggunakan alamat heksadesimal, perbedaan antara *Binary Tree* dan *N-ary Tree*, serta simulasi manual perhitungan Entropy dan Information Gain pada dataset *Play Tennis*. Mahasiswa juga diperkenalkan pada dampak model terhadap keputusan bisnis melalui analisis *Confusion Matrix*.

2. Mindmap



3. Penjelasan Detail

3.1 Representasi Model dalam Memori (RAM & CPU)

Model Machine Learning yang telah dilatih bukan sekadar "file", melainkan struktur data aktif di dalam RAM. Pak Henri menjelaskan bahwa setiap Node dalam pohon keputusan disimpan dalam "kamar" memori dengan alamat spesifik (misal: 13AF, 2C1B).

- **Pointer dan Link:** Setiap simpul menyimpan alamat induknya (Parent) dan daftar alamat anak-anaknya (Children). Jika sebuah simpul tidak memiliki anak lagi, alamat penunjuknya berisi null (0), yang menandakan bahwa simpul tersebut adalah Leaf (daun/hasil akhir).
- **Hierarki Komputasi:** CPU menggunakan L1, L2, dan L3 Cache untuk mempercepat akses perhitungan Tree. Jika pohon terlalu besar, ia akan disimpan di RAM. Inilah mengapa efisiensi model (mengurangi jumlah fitur yang tidak perlu) sangat krusial untuk menjaga performa sistem.

3.2 Kalkulasi Matematis: Langkah Demi Langkah ID3

Algoritma ID3 menggunakan konsep Entropi untuk mengukur "kekacauan" data. Semakin tinggi nilai entropi, semakin sulit data tersebut diprediksi.

- **Menghitung Entropi Total:** Jika kita memiliki 14 data dengan 9 "Yes" dan 5 "No", maka:

$$Entropy(S) = \sum -p_i \log_2 p_i$$

$$Entropy(S) = \left(-\frac{9}{14} \log_2 \frac{9}{14}\right) + \left(-\frac{5}{14} \log_2 \frac{5}{14}\right) \approx 0.940$$

3.3 Menghitung Information Gain

Kita harus mencari fitur yang paling banyak mengurangi entropi. Misalnya pada fitur *Outlook*:

Overcast: Jika 4 data "Overcast" semuanya "Yes", maka entropinya adalah 0 (Data murni).

Sunny/Rainy: Dihitung berdasarkan proporsi masing-masing. Fitur dengan Gain tertinggi akan terpilih menjadi Root (Akar utama).

3.4 Evaluasi dan Risiko Bisnis Model diuji menggunakan Confusion Matri.

Menekankan bahwa kesalahan prediksi memiliki biaya berbeda:

- False Positive: Salah deteksi penipu (Padahal jujur). Risiko: Kehilangan kepercayaan pelanggan.
- False Negative: Gagal deteksi penipu (Padahal maling). Risiko: Kerugian finansial nyata.

4. Sample Code / Pseudocode

Abstraksi Struktur Node untuk Tree di Memori

class DecisionNode:

```
def __init__(self, attribute=None, value=None, results=None, parent=None):
    self.attribute = attribute # Fitur yang diuji (misal: Outlook)
    self.value = value        # Alamat Memori (Hex/Pointer)
    self.results = results     # Nilai jika simpul adalah Leaf
    self.parent = parent      # Link ke Induk
    self.children = {}        # Dictionary simpul anak
```

Pseudocode Algoritma ID3 (Recursive)

def ID3(data, features):

1. Jika semua data labelnya sama, kembalikan Leaf

if all_labels_same(data):

return LeafNode(data[0].label)

2. Hitung Gain untuk setiap fitur

best_feature = max(features, key=lambda f: calculate_gain(data, f))

3. Buat simpul akar dengan fitur terbaik

node = DecisionNode(attribute=best_feature)

4. Ulangi secara rekursif untuk setiap nilai di fitur terbaik

for value in best_feature.values:

subset = data.filter(best_feature == value)

node.children[value] = ID3(subset, features - best_feature)

return node

5. Kesimpulan

Berikut adalah poin-poin kesimpulan detail dari pertemuan tersebut:

5.1 Realitas Fisik Model dalam Memori

Kesimpulan utama mengenai arsitektur model adalah bahwa sebuah Pohon Keputusan (Decision Tree) disimpan di dalam memori (RAM) dalam bentuk sekumpulan simpul (Node) yang saling terhubung.

- Setiap *node* menempati alamat memori tertentu (dalam format Heksadesimal) dan memiliki penunjuk (Pointer) ke alamat simpul induk (*Parent*) maupun simpul anak (*Children*).
- Pemahaman ini krusial karena efisiensi sebuah model (misalnya melalui proses *pruning*) berdampak langsung pada penggunaan L1/L2/L3 Cache CPU. Model yang terlalu besar akan memperlambat proses komputasi, sehingga seorang *AI Engineer* harus mampu menyeimbangkan kompleksitas model dengan kapasitas perangkat keras.

5.2 Inti Algoritma ID3: Reduksi Ambiguitas

Secara matematis, algoritma ID3 disimpulkan sebagai proses pencarian fitur yang paling signifikan secara berulang (*Iterative*). Melalui perhitungan Entropi, kita mengukur tingkat ketidakaturan atau "kekacauan" di dalam data.

- Entropi (H): Semakin tinggi nilainya, semakin ambigu data tersebut.
- Information Gain (IG): Merupakan selisih entropi sebelum dan sesudah data dipecah berdasarkan fitur tertentu. Fitur dengan nilai Gain tertinggi dipilih sebagai akar (Root) karena fitur tersebut memberikan informasi paling murni untuk membedakan antara kelas (misalnya "Yes" atau "No").

5.3 Evaluasi Performa dan Mitigasi Risiko Bisnis

Pertemuan ini menekankan bahwa akurasi tinggi di atas kertas tidak selalu berarti model tersebut "aman" untuk industri. Melalui Confusion Matrix, kita dapat membedah kesalahan model:

- False Positive: Mengakibatkan kerugian berupa hilangnya kepercayaan pelanggan (misal: memblokir transaksi sah karena dianggap *fraud*).
- False Negative: Mengakibatkan kerugian finansial langsung bagi perusahaan (misal: membiarkan transaksi penipuan lolos). Model yang ideal adalah model yang tidak hanya akurat, tetapi juga memiliki tingkat kesalahan yang dapat diterima oleh kebijakan bisnis perusahaan.

5.4 Profesionalisme dan Etika Kerja di Industri

Di dunia industri, Ijazah dan Nilai (A/B) hanyalah tiket masuk, namun *Knowledge* (Pengetahuan Fundamental) adalah penentu keberhasilan.

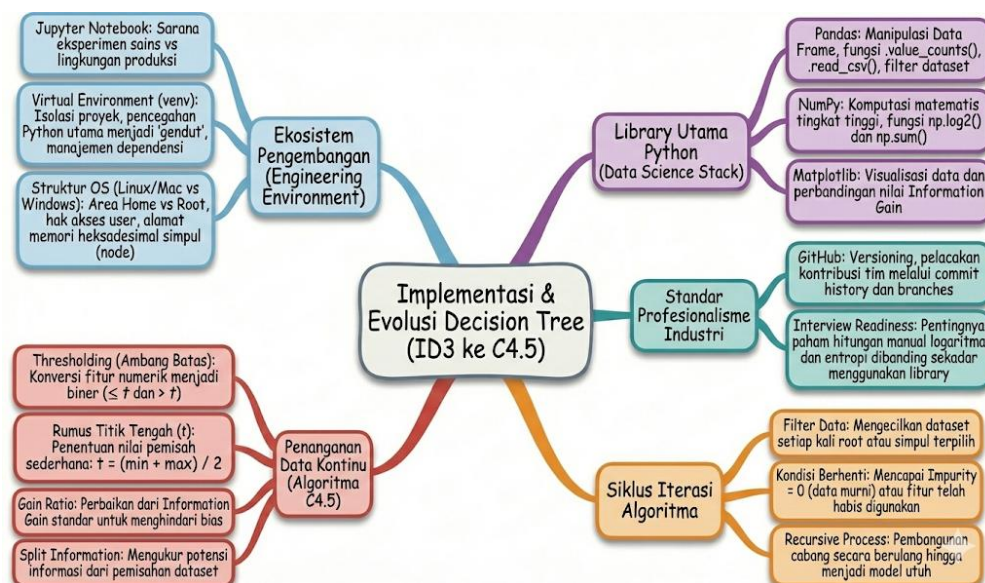
- Mahasiswa diwajibkan menguasai alat kolaborasi standar industri seperti **GitHub**.
- Pemahaman manual (seperti menghitung entropi tanpa bantuan *library*) adalah bukti kestabilan pengetahuan yang dicari oleh perusahaan. Lulus dengan nilai bagus tanpa pemahaman teknis yang kuat hanya akan berujung pada kegagalan saat proses wawancara kerja.

BAB 6 Implementasi Algoritma ID3/C4.5 dalam Python, Penanganan Data Kontinu, dan Best Practice Software Engineering

1. Deskripsi Singkat Pembahasan

Pertemuan keenam memfokuskan pada transisi krusial dari pemahaman teoritis algoritma *Decision Tree* (ID3) menuju implementasi praktis menggunakan bahasa pemrograman Python di ekosistem *Jupyter Notebook*. Bahasan utama mencakup otomatisasi perhitungan Entropy dan Information Gain menggunakan pustaka *Pandas* dan *NumPy*. Selain itu, dibahas perluasan algoritma melalui C4.5 untuk menangani fitur numerik (kontinu) seperti pada dataset *Iris*. Kuliah ini juga menekankan sisi *Engineering*, yaitu pentingnya isolasi proyek menggunakan *Virtual Environment* dan kolaborasi melalui *GitHub* sebagai standar profesional di industri AI.

2. Mindmap



3. Penjelasan Detail

3.1 Otomatisasi Perhitungan ID3 dengan Python

Dalam pertemuan ini, proses manual yang sebelumnya dipelajari dikonversi menjadi fungsi pemrograman. Menggunakan pustaka **Pandas**, kita dapat menghitung frekuensi label dengan metode `.value_counts()`.

- Kalkulasi Entropi: Menghitung proporsi (p_i) dari setiap kelas (misal: 9/14 untuk Yes dan 5/14 untuk No) dan menerapkan fungsi logaritma basis 2 ($np.\log2$).
- Information Gain: Dilakukan dengan mengiterasi setiap fitur, memfilter dataset berdasarkan nilai atribut (misal: membagi data hanya untuk Outlook = Sunny), lalu menghitung penurunan entropinya.

3.2 Penanganan Data Kontinu (Algoritma C4.5)

ID3 memiliki keterbatasan pada data kategori (diskrit). Untuk dataset seperti **Iris** (panjang sepal/petal dalam angka), digunakan algoritma **C4.5**.

- Split Biner: C4.5 mengubah fitur numerik menjadi pemisah biner menggunakan ambang batas (t).
- Penentuan Titik Tengah (t): Metode sederhana yang dibahas adalah dengan menjumlahkan nilai minimum dan maksimum fitur lalu dibagi dua ($\frac{\min + \max}{2}$). Sebagai contoh, pada dataset Iris, ditemukan ambang batas Petal Length sebesar **2.45**. Data dipecah menjadi dua cabang: ≤ 2.45 dan > 2.45 .
- **Gain Ratio**: Untuk menghindari bias pada fitur yang memiliki banyak variasi nilai, C4.5 menggunakan *Gain Ratio* (Information Gain dibagi dengan *Split Information*)

3.3 Pentingnya Isolasi Lingkungan (Virtual Environment)

Penekanan besar pada aspek *Software Engineering*. Menginstal pustaka (seperti *Jupyter* atau *Pandas*) langsung ke sistem operasi (Python utama) dianggap sebagai praktik buruk karena dapat merusak *kernel* sistem.

- **Venv (Virtual Environment):** Menciptakan salinan Python kecil di dalam folder proyek. Hal ini memastikan dependensi antar proyek tidak saling bertabrakan dan menjaga sistem operasi tetap "bersih".
- **Struktur Direktori Linux/Mac:** Dibahas perbandingan hierarki */root* dan */home/user*. Lingkungan pengembangan yang ideal harus berada di dalam direktori *home* agar tidak memerlukan akses *Super Admin* yang berisiko bagi keamanan data.

3.4 Standar Kolaborasi Industri melalui GitHub

Penggunaan GitHub bukan sekadar tempat mengumpulkan tugas, melainkan alat ukur kontribusi anggota kelompok. Melalui commit history dan branches, dosen dapat memverifikasi siapa yang benar-benar melakukan coding dan siapa yang hanya menumpang nama. Pemahaman fundamental matematika tetap menjadi kunci saat wawancara kerja, namun kemampuan menggunakan alat seperti Git adalah syarat mutlak untuk bekerja di tim engineering profesional.

4. Sample Code / Pseudocode

4.1 Fungsi Penghitung Entropi

```
import numpy as np
import pandas as pd

def calculate_entropy(df, target_column):
    # Menghitung proporsi setiap label (Yes/No)
    counts = df[target_column].value_counts()
    probabilities = counts / len(df)

    # Rumus:  $\sum -p_i \cdot \log_2(p_i)$ 
    entropy = -np.sum(probabilities * np.log2(probabilities))
    return entropy
```

4.2 Perhitungan Information Gain untuk Fitur Tertentu

```
def information_gain(df, feature, target_column):
    # 1. Hitung Entropi Awal (S)
    initial_entropy = calculate_entropy(df, target_column)

    # 2. Hitung Entropi Berbobot untuk setiap nilai fitur
    values = df[feature].unique()
    weighted_entropy = 0

    for val in values:
        subset = df[df[feature] == val]
        prob = len(subset) / len(df)
        weighted_entropy += prob * calculate_entropy(subset, target_column)

    # 3. Information Gain = Entropy(S) - Weighted Entropy
    return initial_entropy - weighted_entropy

# Contoh Penggunaan:
# gain_outlook = information_gain(df_tennis, 'Outlook', 'Play Tennis')
```

5. Kesimpulan

1. **Transformasi Skill:** Mahasiswa harus mampu mengubah rumus matematika menjadi fungsi kode yang efisien.
2. **Fleksibilitas Data:** Algoritma C4.5 memungkinkan pohon keputusan memproses data angka melalui mekanisme pemisahan ambang batas (*thresholding*).
3. **Engineering Mindset:** Penggunaan *Virtual Environment* dan *GitHub* adalah pembeda antara "mahasiswa yang sekadar bisa ngoding" dengan seorang "AI Engineer" profesional.
4. **Iterative Process:** Membangun pohon dilakukan selangkah demi selangkah; setelah satu fitur terpilih sebagai *root*, dataset harus difilter untuk menentukan *node* anak berikutnya hingga mencapai kondisi murni (*leaf node*).

Topik : Linear Regression (Single Variate), Analytical vs Numerical Method (SGD) (BAB 7)

1. Deskripsi Singkat Pembahasan

Pada pertemuan ini (Pertemuan ke-6/7), pembahasan difokuskan pada pendalaman algoritma **Linear Regression**. Perkuliahan diawali dengan informasi administratif mengenai Tugas Besar yang mewajibkan mahasiswa membuat paper dengan format Jurnal dan publikasi di GitHub dengan dataset maksimal 10 tahun terakhir.

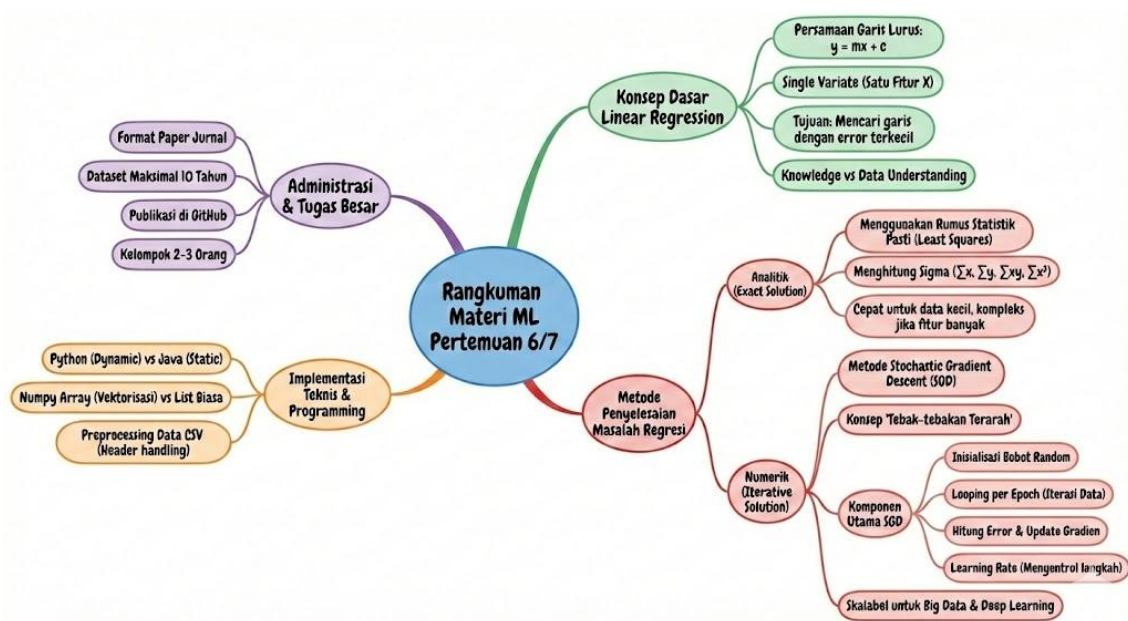
Materi inti membahas dua pendekatan utama dalam menyelesaikan masalah regresi linear:

1. **Pendekatan Analitik (Analytical Solution):** Menggunakan rumus statistik pasti (Least Squares) untuk mencari gradien m dan konstanta c secara langsung.
2. **Pendekatan Numerik (Numerical Solution):** Menggunakan metode iteratif **Stochastic Gradient Descent (SGD)**. Metode ini mencari nilai m dan c terbaik melalui proses *error correction* berulang-ulang (epoch) dengan bantuan *learning rate*.

Selain itu, terdapat pembahasan teknis mengenai implementasi kode (Python vs Java), penggunaan library numpy untuk operasi matriks, dan visualisasi pengaruh jumlah *epoch* terhadap akurasi model garis regresi.

2. Mindmap

Berikut adalah visualisasi pemetaan materi yang dibahas dalam pertemuan ini:



3. Penjelasan Detail Materi

3.1 Linear Regression: Konsep & Pendekatan Analitik

Dasar dari Linear Regression adalah persamaan garis lurus:

$$y = mx + c$$

Atau dalam notasi Machine Learning sering ditulis sebagai:

$$f(x) = w_1x_1 + w_0$$

Dimana w_1 (weight) adalah gradien/kemiringan m dan w_0 (bias) adalah konstanta c .

Tujuan utama algoritma ini adalah mencari garis yang paling "mepet" atau memiliki *error* terkecil terhadap sebaran data. Dalam pendekatan analitik, kita menggunakan rumus statistik yang melibatkan penjumlahan Σ dari elemen x dan y .

- Jika gradien $m = 1$, kemiringan 45 derajat.
- Jika $m > 1$, garis lebih curam.
- Jika $m < 1$, garis lebih landai.

Kelebihan metode analitik adalah hasilnya pasti (exact) dan cepat untuk data kecil. Namun, menjadi kompleks ketika fiturnya bertambah banyak (multivariate).

3.2 Implementasi Teknis: Python, Numpy, dan Perbandingan dengan Java

Dosen memberikan penekanan kuat pada perbedaan implementasi antara Java dan Python.

- **Java (Static Typing & Compiler):** Memerlukan deklarasi tipe data eksplisit dan *boilerplate code* (class, main method) yang panjang. Prosesnya melalui kompilasi menjadi *bytecode*. Java lebih ketat dalam manajemen memori dan tipe data array (ukuran statis).
- **Python (Dynamic Typing & Interpreter):** Lebih ringkas (*straightforward*). Variabel tidak perlu deklarasi tipe. List di Python bersifat dinamis (bisa menampung berbagai tipe data).
- **Numpy:** Untuk kebutuhan *scientific computing*, Python menggunakan library numpy. Array di numpy berbeda dengan list biasa; ia bekerja seperti matriks matematika yang memungkinkan operasi vektor (perkalian seluruh elemen sekaligus tanpa looping manual).

3.3 Stochastic Gradient Descent (SGD)

Materi berlanjut ke pendekatan **Numerik** menggunakan SGD. Ini adalah inti dari pembelajaran mesin modern. Berbeda dengan analitik yang "sekali hitung selesai", SGD melakukan "tebak-tebakan terarah".

Langkah Kerja SGD:

1. **Inisialisasi:** Tentukan nilai awal m dan c secara acak (atau 0).
2. **Iterasi (Epoch):** Lakukan perulangan sebanyak jumlah *epoch* yang ditentukan. Satu epoch berarti model telah melihat seluruh data satu kali.
3. Prediksi & Hitung Error:

Untuk setiap data point, hitung prediksi y' menggunakan m dan c saat ini.

$$Error = y_{prediksi} - y_{asti}$$

4. Update Bobot (Gradient Descent):

Perbarui nilai m dan c agar error mengecil. Rumus pembaruan:

$$m_{baru} = m_{lama} - (LearningRate \times Error \times x)$$

$$c_{baru} = c_{lama} - (LearningRate \times Error)$$

Learning Rate α berfungsi mengontrol seberapa besar langkah perubahan yang diambil (misal 0.01 atau 0.1).

3.4 Eksperimen Epoch dan Konvergensi

Pada bagian akhir, dilakukan eksperimen mengubah jumlah epoch (5.000, 100.000, hingga 1.000.000).

- Epoch sedikit → Garis regresi masih jauh dari titik data (Underfitting).
- Epoch optimal → Garis berada tepat di tengah sebaran data.
- Epoch berlebihan → Memakan waktu komputasi lama namun perbaikan akurasi semakin kecil (diminishing returns) atau bahkan bisa terjadi *overshooting* jika learning rate tidak diatur.

Kesimpulannya, meskipun SGD lebih rumit secara kode dan konsep dibandingkan analitik, SGD adalah metode yang *scalable* untuk data berukuran besar dan dimensi tinggi.

4. Sample Code / Pseudocode

Berikut adalah implementasi kode yang mencerminkan materi perkuliahan, membagi antara cara Analitik (Numpy) dan Numerik (SGD).

4.1 Pendekatan Analitik (Closed Form, Numpy)

Kode ini merepresentasikan perhitungan langsung menggunakan rumus statistik.

Input: data X, Y

Hitung:

$$m = (n \cdot \Sigma(XY) - \Sigma X \cdot \Sigma Y) / (n \cdot \Sigma(X^2) - (\Sigma X)^2)$$

$$c = (\Sigma Y - m \cdot \Sigma X) / n$$

Output: parameter m (gradien) dan c (intercept)

4.2 Pendekatan Numerik (Stochastic Gradient Descent)

Pendekatan ini memperbarui parameter secara iteratif menggunakan gradien error tiap data.

Inisialisasi $m = 0$, $c = 0$

For epoch = 1 sampai jumlah_epoch:

For setiap data (x_i , y_i):

$$y_{\text{pred}} = m \cdot x_i + c$$

$$\text{error} = y_{\text{pred}} - y_i$$

$$m = m - lr \cdot \text{error} \cdot x_i$$

$$c = c - lr \cdot \text{error}$$

Output: parameter m dan c hasil konvergensi

Review Tugas Besar & Pengantar Neural Network

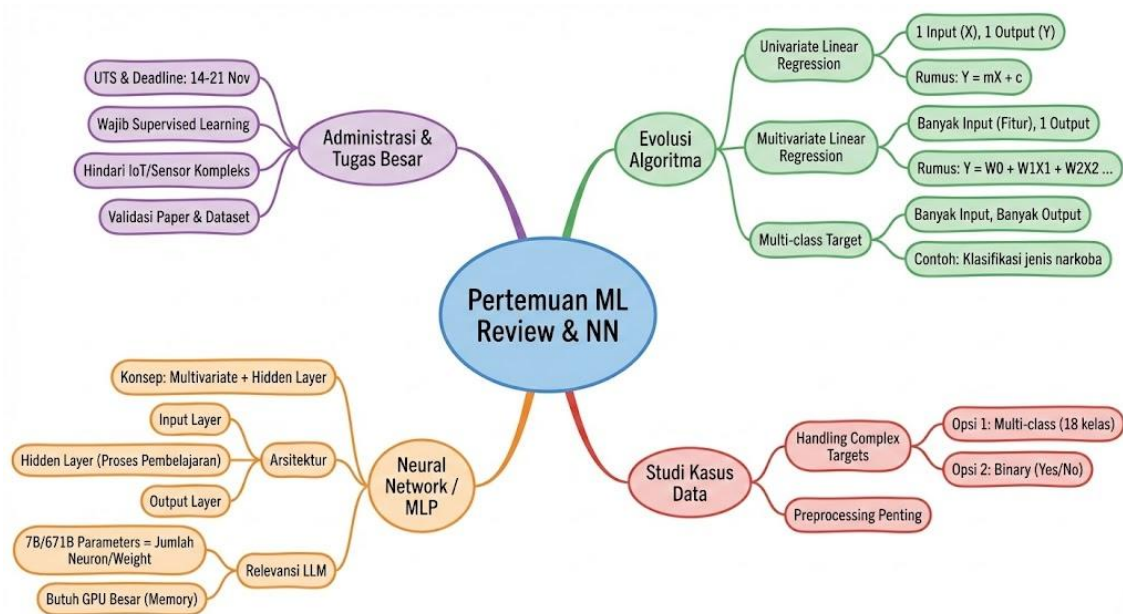
Topik : Transisi Linear Regression ke Neural Network, Multi-class Target, & Evaluasi Proposal (BAB 8)

1. Deskripsi Singkat Pembahasan

Pertemuan ini membahas dua agenda utama: **Review Proposal Tugas Besar** dan **Pendalaman Materi Teknis** dari Linear Regression menuju Neural Network. Dosen menekankan bahwa Tugas Besar wajib menggunakan metode **Supervised Learning**. Banyak kelompok dievaluasi karena memilih metode *Clustering* (Unsupervised) atau memiliki dataset yang tidak jelas target kelasnya.

Secara teknis, materi berkembang dari konsep **Single Variate Linear Regression** menjadi **Multivariate**, hingga terbentuknya arsitektur **Neural Network** dengan penambahan *Hidden Layer*. Dijelaskan pula relevansi materi ini dengan *Large Language Models* (LLM) seperti DeepSeek atau GPT, di mana "Parameter" (misal 7B, 671B) merepresentasikan jumlah *neuron* dan bobot (*weights*) yang saling terhubung.

2. Mind Map



3. Penjelasan Detail & Perumusan Matematis

3.1 Konsep Dasar: Single Variate (Univariate)

Dosen mengulang kembali bentuk paling sederhana dari Linear Regression. Secara visual, ini adalah satu node input terhubung ke satu node output. Persamaan Matematis:

$$f(x) = mx + c$$

Dalam notasi Machine Learning yang digunakan di kelas:

$$y' = w_1 x_1 + w_0 x_0$$

Dimana:

- y' = Output prediksi (misal: Harga Rumah).
- x_1 = Fitur input (misal: Luas Rumah).
- w_1 = *Weight* / Bobot (identik dengan gradien m).
- w_0 = *Bias* (identik dengan konstanta c).
- x_0 = *Bias input* (Selalu bernilai 1, sehingga $w_0 \cdot 1 = w_0$).

3.2 Transisi ke Multivariate Linear Regression

Pembahasan berkembang ke kasus di mana input X lebih dari satu (misal: Luas Rumah, Lokasi, Fasilitas), namun output Y tetap satu.

Persamaan Matematis:

$$y' = (x_0 \cdot w_0) + (x_1 \cdot w_1) + (x_2 \cdot w_2) + \dots + (x_n \cdot w_n)$$

Atau dalam notasi Sigma (penjumlahan):

$$y' = \sum_{i=0}^n w_i x_i$$

Penjelasan:

Dosen menggambarkan garis-garis bobot W_0, W_1, W_2 , yang menghubungkan setiap fitur (X) menuju satu titik output Y . Setiap garis mewakili satu operasi perkalian.

3.3 Neural Network: Multi-class / Multi-output

Dosen menjelaskan bagaimana jika kita memiliki banyak Input X_0 sampai X_n dan banyak Output Y_0 sampai Y_m misalnya untuk klasifikasi jenis narkoba (Sabu, Ganja, Ekstasi).

Setiap *node* output Y memiliki set bobotnya sendiri yang terhubung ke semua *node* input.

Ilustrasi Rumus untuk Output Y_0 :

Dosen mencontohkan perhitungan untuk output pertama Y_0 dengan mengambil bobot spesifik W_1, W_5, W_9, W_{13} yang terhubung padanya:

$$Y_0 = (w_0 \cdot x_0) + (w_1 \cdot x_1) + (w_5 \cdot x_2) + (w_9 \cdot x_3) +$$

Ilustrasi Rumus untuk Output Y_1

Untuk output kedua (Y_1), digunakan set bobot yang berbeda W_2, W_6, W_{10}, W_{14} :

$$Y_1 = (w_2 \cdot x_0) + (w_6 \cdot x_1) + (w_{10} \cdot x_2) + (w_{14} \cdot x_3) +$$

Generalisasi Rumus (Matrix Multiplication):

Secara umum, operasi ini adalah perkalian matriks yang menjadi dasar Neural Network:

$$Y = X \cdot W^T + B$$

Dimana:

- Y adalah vektor output $[y_0, y_1, \dots, y_m]$.
- X adalah vektor input $[x_0, x_1, \dots, x_n]$.
- W adalah matriks bobot di mana w_{ji} menghubungkan input i ke output j .

3.4 Arsitektur Deep Neural Network (Hidden Layer)

Dosen menambahkan *Hidden Layer* di tengah arsitektur. Perhitungannya dilakukan secara bertahap (*Forward Propagation*).

1. Hitung Hidden Layer (H):

$$h_j = \sum (x_i \cdot w_{ij}^{(1)})$$

(Input dikali bobot layer pertama).

2. Hitung Output Layer (O):

$$o_k = \sum (h_j \cdot w_{jk}^{(2)})$$

(Hasil Hidden Layer dikali bobot layer kedua).

Inilah yang dimaksud dengan "**7 Billion Parameters**" pada model LLM (DeepSeek/GPT); angka tersebut adalah jumlah total *w* (bobot) dalam jutaan persamaan seperti di atas yang harus dihitung komputer.

4. Kesimpulan

Berdasarkan pembahasan pada pertemuan ini, dapat disimpulkan bahwa:

1. **Fundamental Neural Network:** Neural Network bukanlah "sihir", melainkan evolusi matematis dari **Multivariate Linear Regression**. Dengan menambahkan *Hidden Layer* di antara *Input* dan *Output*, model mampu mempelajari pola yang jauh lebih kompleks dibandingkan regresi linear biasa.
2. **Demistifikasi AI Generatif:** Istilah parameter pada model besar (seperti DeepSeek 7B atau 671B) secara harfiah merujuk pada jumlah *neuron* dan *bobot* (weights) dalam arsitektur Neural Network tersebut. Hal ini menegaskan bahwa GenAI hanyalah kalkulasi statistik probabilitas dalam skala masif yang membutuhkan komputasi (GPU) tinggi.
3. **Kritikalitas Preprocessing Data:** Dalam *Supervised Learning*, kejelasan **Target Class** adalah mutlak. Mahasiswa harus mampu membedah dataset yang kompleks (seperti kasus 18 kelas target pada data konsumsi obat) menjadi format yang dapat dipelajari mesin, baik melalui penyederhanaan menjadi *Binary Classification* atau tetap *Multi-class* dengan strategi yang tepat.

Transisi Linear Regression ke Neural Network & Feed Forward

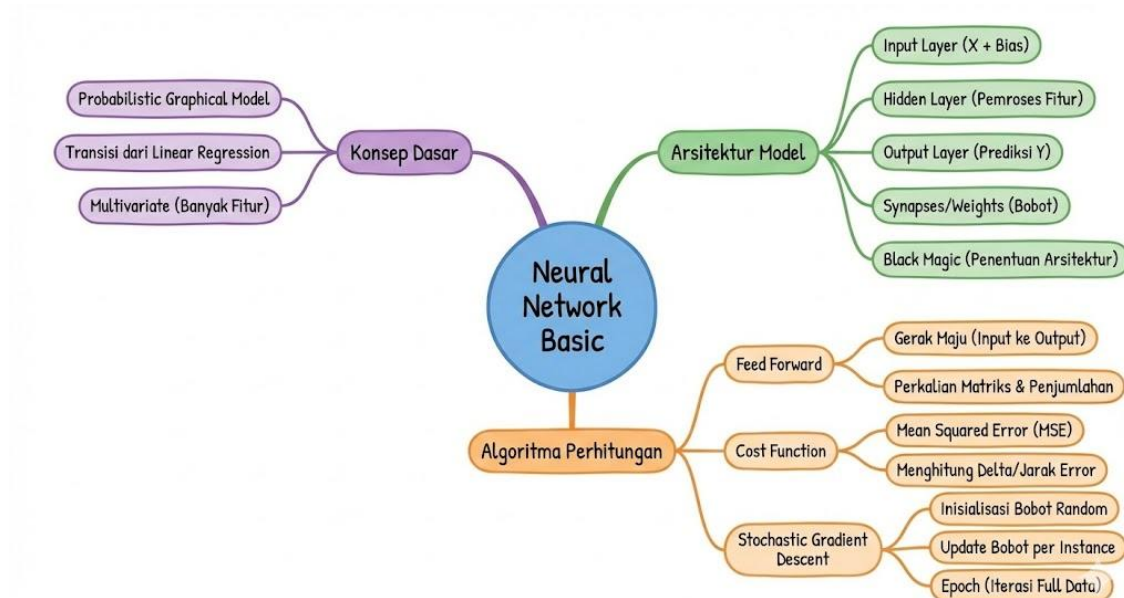
Topik : Multivariate Regression, Hidden Layer Architecture, & Stochastic Gradient Descent (Feed Forward) (BAB 9)

1. Deskripsi Singkat Pembahasan

Pertemuan ini membahas evolusi dari **Linear Regression** (Statistik) menuju **Probabilistic Graphical Model** (Neural Network). Fokus utama adalah transformasi dari *Single Variate* menjadi **Multivariate Linear Regression**, yang kemudian dikembangkan dengan penambahan **Hidden Layer** untuk membentuk Neural Network sederhana.

Materi mencakup teknis perhitungan **Feed Forward**, yaitu proses data bergerak dari Input ke Output melalui perkalian bobot (*weights*), perhitungan *error* menggunakan **Mean Squared Error (MSE)**, dan konsep dasar pembaruan bobot menggunakan **Stochastic Gradient Descent (SGD)**. Dijelaskan pula tantangan penentuan arsitektur model yang sering disebut sebagai "*Black Magic*" karena membutuhkan eksperimen empiris.

2. Mind Map



3. Penjelasan Detail & Perumusan Matematis

3.1 Multivariate Linear Regression

Pada konsep *univariate*, kita hanya menggunakan satu fitur x . Dalam *multivariate*, kita menggunakan banyak fitur $x_1, x_2, \dots, x_n$ ditambah dengan **Bias** (x_0 yang selalu bernilai 1).

Persamaan Matematis:

$$f(x) = w_0x_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

Atau dalam notasi Sigma (Summation):

$$f(x) = \sum_{i=0}^n w_i x_i$$

Dimana:

- x_0 = Bias input (bernilai 1).
- w_0 = Bias weight (bobot bias).
- x_i = Fitur input ke- i .
- w_i = Bobot untuk fitur ke- i .

3.2 Arsitektur Neural Network (Hidden Layer)

Untuk menangani kompleksitas data yang tidak linear, ditambahkan lapisan di antara *Input* dan *Output* yang disebut Hidden Layer.

- **Input Layer:** Tempat data masuk (termasuk bias).
- **Hidden Layer:** Lapisan pemroses di tengah. Jumlah *neuron* dan *layer* di sini disebut sebagai "*Black Magic*" karena penentuannya tidak ada rumus pasti, melainkan melalui eksperimen (seperti mendesain roket ke bulan: butuh bahan bakar/data dan desain roket/arsitektur yang tepat).
- **Output Layer:** Hasil prediksi.

Notasi Perhitungan Node pada Hidden Layer:

Jika N_j adalah neuron pada hidden layer, maka nilainya adalah jumlah perkalian input dengan bobot yang masuk ke neuron tersebut:

$$N_j = \sum (Input_i \cdot w_{ij})$$

Contoh untuk Neuron 0 (N_0) pada Hidden Layer:

$$N_0 = (x_0 \cdot w_{00}) + (x_1 \cdot w_{10}) + (x_2 \cdot w_{20}) +$$

3.3 Algoritma Feed Forward & Cost Function

Feed Forward adalah proses menghitung nilai dari input, melewati hidden layer, hingga menghasilkan output prediksi (y' atau O). Algoritma ini hanya berjalan "maju".

Setelah mendapatkan output prediksi (y'), kita harus menghitung seberapa salah prediksi tersebut dibandingkan target asli (y). Ini disebut **Delta** atau **Error**.

Rumus Delta (Error):

$$\delta = y_{target} - y_{prediction}$$

Untuk mengukur performa keseluruhan model dalam satu *epoch* (satu putaran penuh data), digunakan **Mean Squared Error (MSE)** atau Cost Function (J):

$$MSE = \frac{1}{2n} \sum_{i=1}^n (y_i - y'_i)^2$$

- Dikudratkan agar nilai negatif menjadi positif.
- Dibagi $2n$ untuk memudahkan penurunan rumus (derivatif) nantinya.

3.4 Stochastic Gradient Descent (SGD) - Weight Update

Setelah tahu besaran error, kita harus memperbarui bobot (W) agar error mengecil. Inilah inti dari "belajar". SGD melakukan update bobot per data (*instance*) atau per *batch*.

Rumus Update Bobot Sederhana:

$$W_{new} = W_{old} + \alpha \cdot \text{gradient}$$

Dimana α adalah **Learning Rate** (misal 0.1). Secara lebih detail dengan momentum:

$$W_{ij(new)} = W_{ij(old)} + (\alpha \cdot \delta \cdot x_{input})$$

- Jika error positif, bobot ditambah.
- Jika error negatif, bobot dikurangi.

4. Pseudocode (Feed Forward Logic)

Berikut adalah logika implementasi *Feed Forward* dan perhitungan error tanpa menggunakan library kalkulasi otomatis.

Inisialisasi

INPUT_NODES = 3 (+1 Bias)

HIDDEN_NODES = 3

OUTPUT_NODES = 1

EPOCHS = 100

LEARNING_RATE = 0.1

```
# 1. Inisialisasi Bobot secara Random
weights_input_hidden = random_matrix(INPUT_NODES, HIDDEN_NODES)
weights_hidden_output = random_matrix(HIDDEN_NODES, OUTPUT_NODES)

# 2. Loop Training (Epoch)
for epoch in range(EPOCHS):
    total_error = 0

# 3. Loop per Data Point (Stochastic)
for data in dataset:
    # --- FEED FORWARD ---
    # Hitung nilai Hidden Layer
    hidden_vals = []
    for j in range(HIDDEN_NODES):
        val = sum(data[i] * weights_input_hidden[i][j] for i in range(INPUT_NODES))
        hidden_vals.append(val)

    # Hitung nilai Output Layer
    y_pred = sum(hidden_vals[j] * weights_hidden_output[j] for j in range(HIDDEN_NODES))

    # --- HITUNG ERROR ---
    y_actual = data_target
    error = y_actual - y_pred

    # Akumulasi MSE untuk monitoring
    total_error += (error ** 2)

    # --- UPDATE WEIGHT (Sederhana) ---
    # (Detail Backpropagation akan dibahas pertemuan selanjutnya)
    update_weights_based_on_error(error, LEARNING_RATE)

# Hitung rata-rata error per epoch
mse = total_error / (2 * len(dataset))
print(f"Epoch {epoch}: MSE = {mse}")
```

5. Kesimpulan

Berdasarkan materi pertemuan ini, dapat disimpulkan bahwa:

1. **Fundamental Neural Network:** Neural Network sejatinya adalah pengembangan dari Multivariate Linear Regression yang disusun secara bertingkat (*layering*). Setiap koneksi antar neuron merepresentasikan satu variabel bobot (w) yang harus dicari nilainya.
2. **Pentingnya Hidden Layer:** Hidden layer memberikan kemampuan pada model untuk memecahkan masalah non-linear. Jumlah neuron dan layernya ditentukan melalui eksperimen (*trial & error*) yang disebut sebagai "Black Magic" atau pencarian arsitektur.
3. **Mekanisme Belajar:** Proses belajar mesin terdiri dari dua tahap utama:
 - **Feed Forward:** Menghitung prediksi dari depan ke belakang.
 - **Error Calculation (MSE):** Menghitung selisih prediksi dengan kenyataan.
 - **Weight Update (SGD):** Mengoreksi bobot berdasarkan error tersebut agar prediksi selanjutnya lebih akurat.
4. **Hardware Dependency:** Karena melibatkan jutaan hingga miliaran operasi perkalian matriks (terutama pada Deep Learning), penggunaan **GPU** menjadi mutlak diperlukan dibandingkan CPU biasa karena kemampuan pemrosesan paralelnya.

Neural Network Architecture & Data Preprocessing

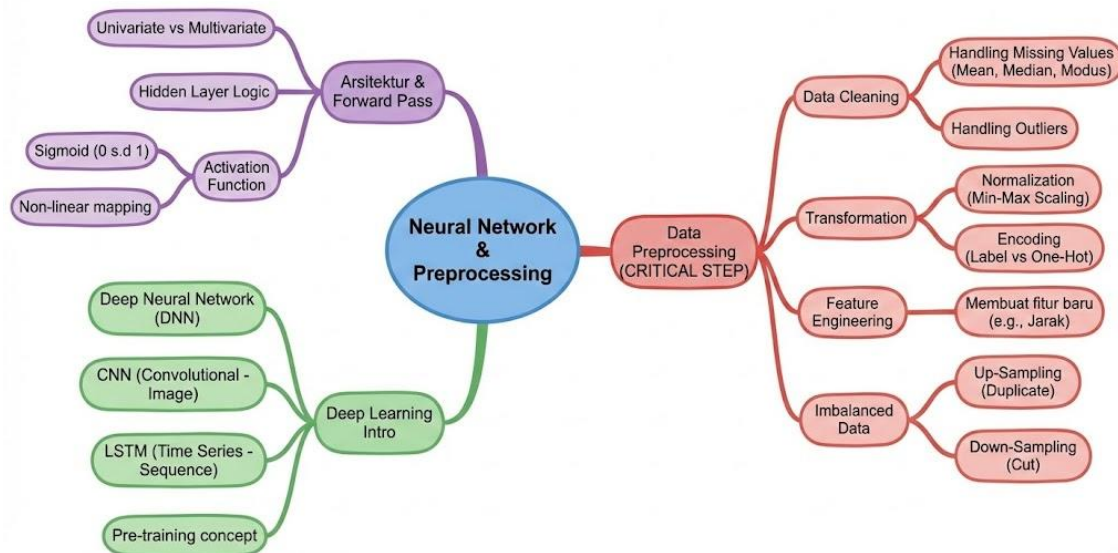
Topik : Feed Forward, Fungsi Aktivasi Sigmoid, & Teknik Preprocessing Data (BAB 10)

1. Deskripsi Singkat Pembahasan

Pertemuan ini memperdalam transisi dari **Multivariate Linear Regression** menuju **Neural Network** dengan penambahan fungsi aktivasi **Sigmoid**. Fokus utama bergeser pada pentingnya **Data Preprocessing** sebelum *training* model, mengingat data dunia nyata (seperti data transaksi *fraud*) seringkali kotor, tidak seimbang (*imbalanced*), dan memiliki skala yang berbeda.

Selain itu, dibahas pengantar **Deep Learning** sebagai evolusi Neural Network dengan arsitektur yang lebih kompleks, seperti **CNN** (untuk citra), **LSTM** (untuk *time series*), dan **GAN**. Dosen juga mengingatkan tenggat waktu Tugas Besar yang dimajukan ke tanggal 21 November.

2. Mind Map



3. Penjelasan Detail & Perumusan Matematis

3.1 Feed Forward & Fungsi Sigmoid

Pada *Multivariate Linear Regression*, output (y) dihasilkan dari penjumlahan bobot dikali input. Dalam Neural Network, hasil penjumlahan ini sering kali perlu dipetakan ke rentang tertentu (misal 0 sampai 1 untuk probabilitas) menggunakan fungsi aktivasi.

1. Rumus Dasar (Linear Sum):

$$z = \sum_{i=0}^n w_i x_i = (w_0 x_0) + (w_1 x_1) + \dots + (w_n x_n)$$

2. Fungsi Aktivasi Sigmoid:

Dosen menjelaskan penggunaan Sigmoid untuk mengubah nilai linear menjadi non-linear (0 s.d 1).

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sehingga output neuron menjadi:

$$Output = \sigma \left(\sum w_i x_i \right)$$

3.2 Notasi Layer Neural Network

Untuk menangani Hidden Layer, notasi indeks menjadi krusial.

Jika $N_{layer,neuron}$ adalah neuron, dan $W_{input,neuron}$ adalah bobot:

$$N_{1,1} = \sigma \left(\sum (X_i \cdot W_{i,1}) \right)$$

Dimana input X dikalikan dengan bobot yang menuju ke neuron 1 di layer 1.

3.3 Data Preprocessing

Data mentah tidak bisa langsung masuk ke model. ada beberapa teknik vital:

1. Normalization (Min-Max Scaling):

Data dengan range berbeda (misal: Gaji jutaan vs Umur puluhan) akan menyulitkan model mencapai konvergen. Solusinya adalah mengubah semua data ke range [0, 1].

Rumus Min-Max Scaling:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Contoh Kasus: Gaji 15 Juta, dengan Range Min 1 Juta dan Max 100 Juta.

$$X_{norm} = \frac{15 - 1}{100 - 1} = \frac{14}{99} \approx 0.14$$

2. Encoding (Data Kategorikal ke Numerik):

- **Label Encoding:** Mengubah kategori menjadi angka urut (Fraud=0, Suspect=1, Non-Fraud=2). Kelemahan: Model bisa salah mengartikan urutan angka sebagai tingkatan nilai.
- **One-Hot Encoding:** Membuat kolom baru (dummy variables) untuk setiap kategori agar biner (0/1). Contoh: Kolom Is_Fraud, Is_Suspect, Is_Legit.

3. Imbalanced Data Handling:

Kasus Fraud biasanya sangat sedikit (misal 1%) dibanding transaksi normal.

- **Up-Sampling:** Menduplikasi data minoritas (Fraud) agar jumlahnya setara.
- **Down-Sampling:** Memotong data mayoritas (Normal) agar jumlahnya turun setara dengan data fraud.

4. Implementation Code (Python)

Berikut adalah implementasi kode untuk **Min-Max Normalization**, **Sigmoid Function**, dan **One-Hot Encoding** sesuai pembahasan teknis dosen.

1. Min-Max Normalization

Input: data X

$$X_norm = (X - X_min) / (X_max - X_min)$$

Output: data ternormalisasi (rentang 0–1)

2. One-Hot Encoding

Input: label kategorikal Y

Untuk setiap data y_i :

Jika $y_i == kelas_j \rightarrow 1$

Selain itu $\rightarrow 0$

Output: matriks biner hasil encoding

3. Sigmoid Activation & Forward Pass

Input: vektor input x , bobot w , bias b

$$z = x \cdot w + b$$

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

Output: nilai aktivasi neuron

5. Kesimpulan

Berdasarkan materi pertemuan ini, poin-poin krusial yang dapat disimpulkan adalah:

1. **Garbage In, Garbage Out:** Sebagus apapun arsitektur Neural Network (bahkan Deep Learning sekalipun), jika data mentah tidak melalui **Preprocessing** yang benar (Normalisasi, Cleaning, Encoding), model akan sulit konvergen atau bias.
2. **Peran Fungsi Aktivasi:** Penambahan **Sigmoid** pada *Linear Regression* adalah kunci transformasi menjadi *Neural Network*, memungkinkan model menangani probabilitas dan data non-linear.
3. **Strategi Data Tidak Seimbang:** Pada kasus dunia nyata seperti deteksi *fraud*, teknik *Up-sampling* atau *Down-sampling* mutlak diperlukan agar model tidak bias ke kelas mayoritas.
4. **Evolusi ke Deep Learning:** Neural Network biasa (MLP) adalah pondasi. Ketika *Hidden Layer* diperbanyak dan ditambahkan arsitektur khusus (seperti filter pada CNN atau memori pada LSTM), ia menjadi **Deep Learning** yang mampu melakukan ekstraksi fitur secara otomatis (mengurangi kebutuhan *feature engineering* manual, meskipun *preprocessing* dasar tetap disarankan).

Machine Learning Engineering & Unsupervised Learning

Topik : ML Pipeline, Deep Learning Intro, & Clustering Algorithms (K-Means vs GMM) (BAB 11)

1. Deskripsi Singkat Pembahasan

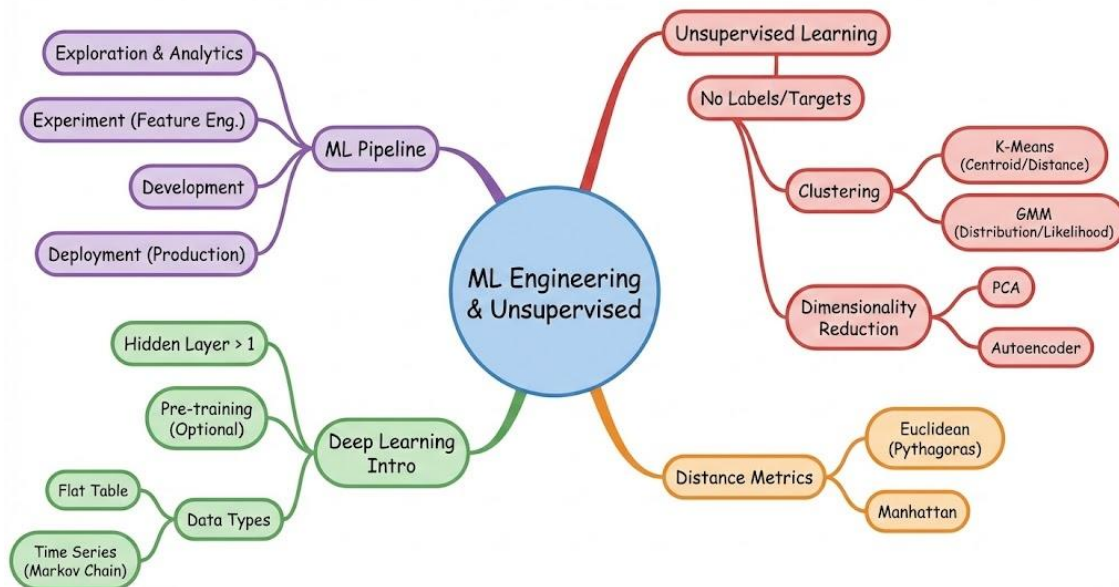
Pertemuan ini membahas transisi dari *Science* (menjawab pertanyaan "mengapa") ke *Engineering* (membangun solusi). Fokus utama adalah **Machine Learning Pipeline**, mulai dari *Data Exploration* hingga *Deployment*.

Secara teknis, materi beralih ke **Unsupervised Learning**, yaitu pembelajaran mesin pada data tanpa label (*unlabeled*). Dibahas dua pendekatan utama dalam *Clustering*:

1. **K-Means**: Berbasis jarak (*distance*) menggunakan *Centroid*.
2. Gaussian Mixture Models (GMM): Berbasis probabilitas distribusi data (*likelihood*).

Selain itu, dibahas konsep Dimensionality Reduction untuk menangani data berdimensi tinggi yang sulit divisualisasikan manusia (terbatas pada 3D).

2. Mind Map



3. Penjelasan Detail & Perumusan Matematis

3.1 Machine Learning Pipeline

Proses *engineering* dalam ML tidak hanya *modeling*. Alurnya adalah:

1. **Exploration & Analytics**: Memahami distribusi statistik data.
2. **Experiment**: *Preprocessing*, *Feature Engineering* (misal: mengekstrak fitur "Jarak" dari kode pos), dan *Modeling*.
3. **Development & Deployment**: Membawa model ke *production* (misal: via Docker/API).

3.2 Unsupervised Learning: Clustering

Berbeda dengan *Supervised Learning* (ada target y), *Unsupervised* bertujuan menemukan pola tersembunyi atau pengelompokan data.

1. K-Means Clustering

Algoritma ini mengelompokkan data berdasarkan jarak terdekat ke titik pusat (*Centroid*). K adalah jumlah *cluster* yang ditentukan di awal.

- **Prinsip:** Iteratif menggeser *centroid* hingga posisi stabil.
- **Metode Perhitungan Jarak:** Menggunakan **Euclidean Distance** (Teorema Pythagoras).

Rumus Euclidean Distance (2 Dimensi):

$$d(p, q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2}$$

Rumus Euclidean General (Multi-dimensi):

$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

Dimana p dan q adalah dua titik data, dan n adalah jumlah fitur/dimensi.

2. Gaussian Mixture Models (GMM)

Berbeda dengan K-Means yang kaku (bulat), GMM mengasumsikan data berkumpul membentuk distribusi normal ("gunung").

- **Prinsip:** Probabilistik. Setiap *cluster* dimodelkan sebagai distribusi Gaussian.
- **Metrik:** Menggunakan **Likelihood** (seberapa besar kemungkinan data masuk ke distribusi tertentu), bukan *error* jarak.

3. Perbedaan K-Means vs KNN

Dosen menekankan agar tidak tertukar:

- **K-Means (Unsupervised):** Mengelompokkan data tanpa label. K = jumlah cluster.
- **KNN (Supervised - Lazy Learning):** Klasifikasi berdasarkan tetangga terdekat. K = jumlah tetangga yang divoting. KNN boros komputasi karena menghitung jarak setiap kali ada data baru (*inference*).

3.3 Dimensionality Reduction

Manusia hanya mampu memvisualisasikan hingga 3 dimensi (X, Y, Z). Data riil bisa memiliki ratusan dimensi.

- **Masalah:** *Curse of Dimensionality*.
- **Solusi:** Algoritma seperti **PCA (Principal Component Analysis)** atau **Autoencoder** digunakan untuk mereduksi dimensi (misal 100 fitur $\rightarrow$ 3 fitur utama) dengan mempertahankan informasi sebanyak mungkin, agar pola data (topologi) tetap terlihat.

4. Implementation Code (K-Means Clustering)

Berikut adalah implementasi K-Means menggunakan Python (sklearn) untuk mendemonstrasikan pengelompokan data berdasarkan jarak Euclidean, sesuai pembahasan teori.

Input: data X, jumlah cluster K

Inisialisasi centroid awal

Ulangi sampai konvergen:

 Hitung jarak Euclidean setiap data ke tiap centroid

 Kelompokkan data ke centroid terdekat

 Perbarui posisi centroid (rata-rata anggota cluster)

Output: label cluster dan posisi centroid

Alternatif: Versi pakai sklearn

```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=2)  
kmeans.fit(X)
```

```
labels = kmeans.labels_  
centroids = kmeans.cluster_centers_
```

5. Kesimpulan

Dari pertemuan ini, dapat disimpulkan:

1. **Evolusi Peran:** Seorang *Machine Learning Engineer* harus menguasai *end-to-end pipeline*, tidak hanya pemodelan, tetapi juga pemahaman bisnis, *preprocessing*, hingga *deployment*.
2. **Paradigma Unsupervised:** Ketika data tidak memiliki label, kita menggunakan *Clustering*. Pemilihan algoritma bergantung pada bentuk data: gunakan **K-Means** jika data terpisah secara geometris (jarak), dan gunakan **GMM** jika data bertumpuk dalam distribusi probabilitas.
3. **Konsep Jarak & Dimensi:** Inti dari banyak algoritma ML (K-Means, KNN) adalah perhitungan jarak (*Euclidean*). Tantangan muncul pada data berdimensi tinggi yang memerlukan teknik *Dimensionality Reduction* agar dapat diproses dan divisualisasikan.
4. **Tugas Besar:** Mahasiswa wajib memahami perbedaan *Supervised* (untuk UTS) dan *Deep Learning* (untuk UAS) serta memastikan dataset yang dipilih relevan untuk kedua metode tersebut.

Review Tugas & Algoritma Clustering Lanjutan

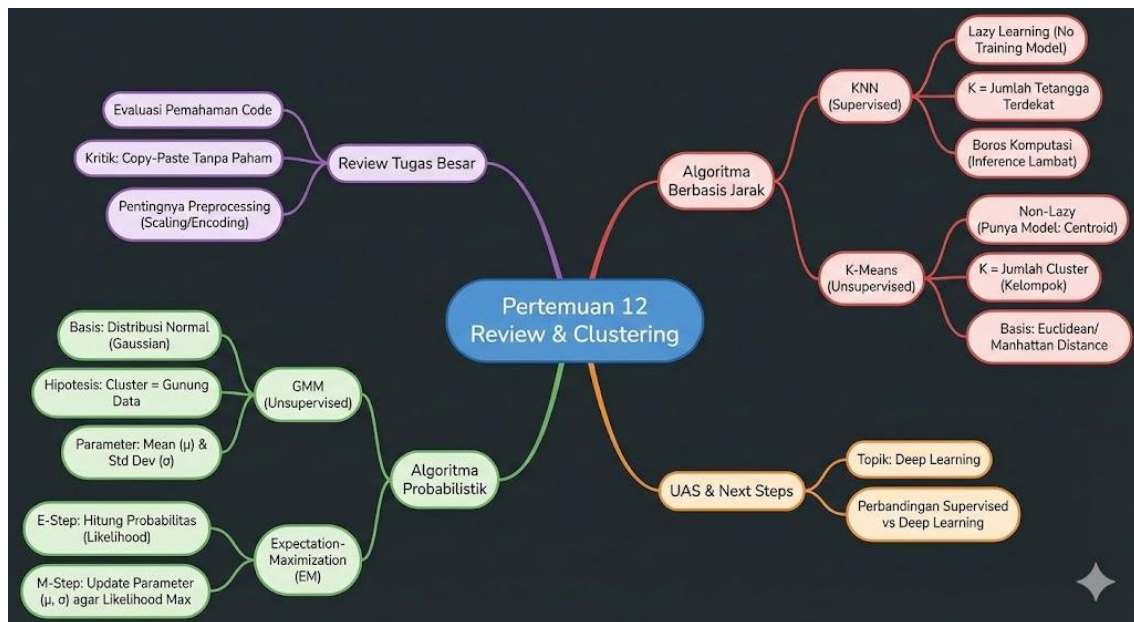
Topik: Evaluasi Supervised Learning, KNN vs K-Means, & Gaussian Mixture Model (GMM) (BAB 12)

1. Deskripsi Singkat Pembahasan

Pertemuan ini diawali dengan **Review Tugas Besar (UTS)** di mana dosen melakukan pengecekan acak terhadap pemahaman mahasiswa mengenai kode program (*notebook*) yang dikumpulkan. Banyak mahasiswa ditegur karena hanya menyalin kode tanpa memahami fungsi krusial seperti *StandardScaler*, *Encoding*, atau visualisasi data.

Sesi materi berlanjut membahas perbedaan fundamental antara **KNN (Supervised/Lazy)** dan **K-Means (Unsupervised/Clustering)** yang sering tertukar karena notasi "K". Pembahasan teknis mendalam difokuskan pada **Gaussian Mixture Model (GMM)** yang menggunakan pendekatan probabilistik (distribusi normal/gunung) untuk *clustering*, berbeda dengan K-Means yang berbasis jarak (*distance*). Algoritma **Expectation-Maximization (EM)** dijelaskan sebagai "mesin" di balik GMM untuk mencari parameter distribusi (Mean μ dan Covariance Σ) yang paling cocok (*likelihood*) dengan data.

2. Mind Map



3. Penjelasan Detail & Perumusan Matematis

3.1 Review Tugas: *Critical Thinking* dalam Coding

Dosen menekankan bahwa menggunakan AI untuk *coding* diperbolehkan, asalkan mahasiswa paham alur logikanya. Beberapa poin teknis yang disorot:

- **Encoding:** Mengubah data kategorikal (Teks) menjadi numerik (0/1).
- **Standard Scaler:** Menormalisasi data agar memiliki *mean* 0 dan *std dev* 1, penting untuk algoritma berbasis jarak (SVM, KNN).
- **Confusion Matrix:** Cara membaca akurasi prediksi vs data aktual.

3.2 KNN (Supervised) vs K-Means (Unsupervised)

Dua algoritma ini sering membingungkan karena sama-sama menggunakan huruf "K".

Fitur	KNN (K-Nearest Neighbors)	K-Means Clustering
Jenis	Supervised (Punya Label Target)	Unsupervised (Tidak Punya Label)
Arti K	Jumlah Tetangga terdekat untuk voting.	Jumlah Cluster/Kelompok yang ingin dibentuk.
Sifat	<i>Lazy Learning</i> (Tidak training model di awal, hitung saat ada data baru).	<i>Eager Learning</i> (Training mencari Centroid).
Output	Prediksi Kelas berdasarkan mayoritas tetangga.	Posisi Centroid & Anggota Cluster.

Konsep Jarak (Euclidean Distance):

Keduanya menggunakan rumus jarak untuk menentukan kedekatan.

$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

3.3 Gaussian Mixture Model (GMM) & Expectation-Maximization (EM)

Jika K-Means memisahkan data berdasarkan **Jarak Geometris** (bulat kaku), GMM memisahkan data berdasarkan **Distribusi Probabilitas** (bentuk lonjong/gunung).

1. Konsep Dasar GMM:

Setiap cluster diasumsikan mengikuti Distribusi Normal (Gaussian).

$$N(x|\mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- **μ (Mu):** Rata-rata (pusat puncak gunung).
- **σ (Sigma):** Standar Deviasi (lebar/sebaran gunung).

2. Algoritma EM (Expectation-Maximization):

Karena kita tidak tahu parameter μ dan σ untuk data tanpa label, kita gunakan EM:

- **E-Step (Expectation):** Menebak probabilitas setiap data masuk ke cluster tertentu (Soft Clustering).
 - *Rumus:* Menghitung *Likelihood*.
- **M-Step (Maximization):** Memperbarui parameter μ dan σ berdasarkan data yang sudah "ditebak" di E-Step agar *Likelihood* total menjadi maksimal.
 - *Update Mu:* Rata-rata tertimbang dari data.
 - *Update Sigma:* Variansi tertimbang dari data.

Proses ini berulang (Iteratif) hingga parameter konvergen (stabil), mirip seperti K-Means menggeser *centroid*.

4. Implementation Code (Gaussian Mixture Model)

Berikut pseudocode singkat implementasi Gaussian Mixture Model (GMM) untuk clustering probabilistik menggunakan algoritma Expectation–Maximization (EM).

Input: data X, jumlah komponen K

Inisialisasi parameter awal (mean, covariance, bobot)

Ulangi sampai konvergen:

E-step

Hitung probabilitas tiap data berasal dari setiap cluster

M-step

Perbarui mean, covariance, dan bobot berdasarkan probabilitas

Output:

label cluster (argmax probabilitas)

probabilitas keanggotaan (soft clustering)

Versi Semi-Kode

```
from sklearn.mixture import GaussianMixture
```

```
gmm = GaussianMixture(n_components=2)
```

```
gmm.fit(X)
```

```
labels = gmm.predict(X)
```

```
probs = gmm.predict_proba(X)
```

5. Kesimpulan

Dari pertemuan ini, dapat disimpulkan:

1. **Pentingnya Pemahaman Code:** Dalam tugas besar maupun dunia kerja, kemampuan menjelaskan *mengapa* suatu kode/algoritma digunakan (misal: alasan pakai *StandardScaler*) jauh lebih dihargai daripada sekadar kode yang berjalan.
2. **KNN vs K-Means:** Jangan tertukar. KNN untuk klasifikasi (Supervised) berbasis tetangga, K-Means untuk pengelompokan (Unsupervised) berbasis *centroid*.
3. **Fleksibilitas GMM:** GMM adalah algoritma *clustering* yang lebih fleksibel daripada K-Means. GMM menggunakan probabilitas (*Likelihood*), sehingga bisa menangani data yang *overlap* atau berbentuk lonjong, serta memberikan *soft clustering* (tingkat keyakinan).
4. **Persiapan UAS:** Materi UAS akan berfokus pada **Deep Learning**. Mahasiswa wajib memahami konsep dasar Machine Learning saat ini sebagai fondasi untuk memahami arsitektur Deep Learning yang lebih kompleks nanti.

Probability in Clustering & Introduction to Tensors

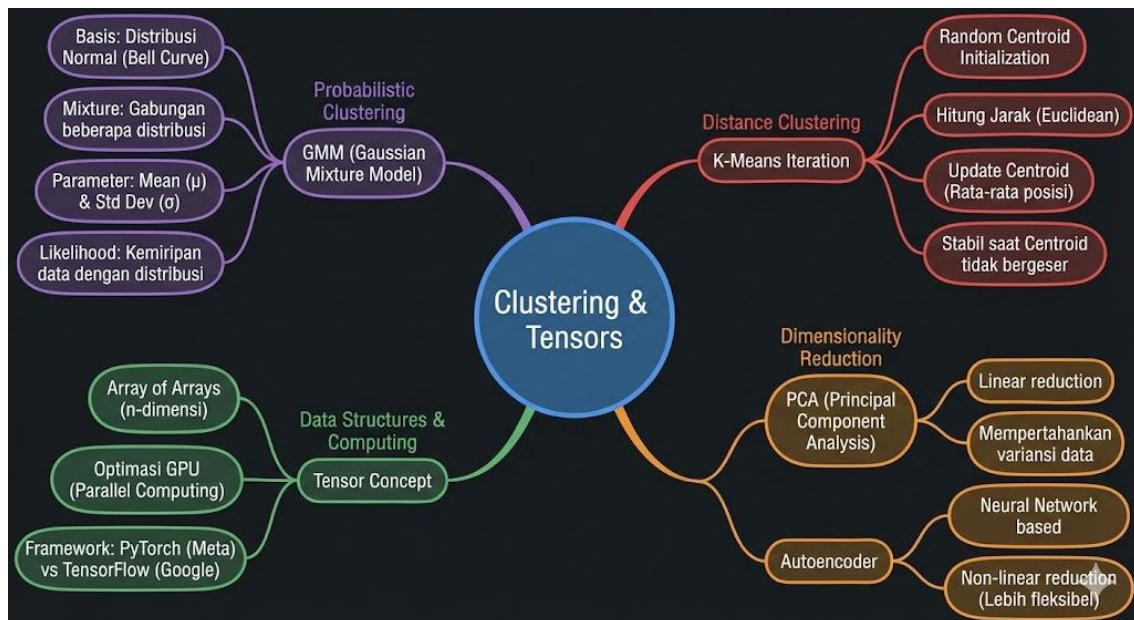
Topik: GMM Probability, K-Means Iteration, Tensor Operations (PyTorch), & Dimensionality Reduction (BAB 13)

1. Deskripsi Singkat Pembahasan

Pertemuan ini membahas secara mendalam mekanisme matematis di balik algoritma *Clustering*, khususnya **Gaussian Mixture Models (GMM)** yang menggunakan pendekatan probabilistik (distribusi normal) dibandingkan **K-Means** yang berbasis jarak (*centroid*). Diskusi mencakup bagaimana GMM menghitung kemungkinan (*likelihood*) suatu data masuk ke dalam kelompok tertentu menggunakan parameter **Mean (μ)** dan **Standard Deviation (σ)**.

Selain itu, diperkenalkan konsep **Tensor** sebagai struktur data utama dalam *Deep Learning* yang memungkinkan komputasi paralel pada **GPU** (menggunakan *framework* seperti PyTorch). Dibahas pula teknik **Dimensionality Reduction** (PCA vs Autoencoder) untuk memvisualisasikan data berdimensi tinggi ke dalam bentuk 2D/3D agar dapat dianalisis oleh manusia.

2. Mind Map



3. Penjelasan Detail & Perumusan Matematis

3.1 GMM: Probability & Gaussian Distribution

Berbeda dengan K-Means yang membagi data secara kaku (Hard Clustering), GMM melihat "gunung" data. Jika data berkumpul membentuk bukit, itu adalah satu *cluster*.

1. Probability Density Function (PDF) Gaussian:

Untuk menghitung seberapa mungkin data x masuk ke cluster tertentu:

$$P(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

- **μ (Mu):** Titik tengah puncak gunung (rata-rata).
- **σ (Sigma):** Lebar gunung (sebaran/simpangan baku).

2. Mixture Model:

Jika ada K cluster (gunung), maka total probabilitas adalah jumlah dari semua kemungkinan tersebut yang dibobotkan (π):

$$P(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$$

3.2 K-Means Iteration Logic

K-Means mencari *centroid* secara iteratif.

1. Inisialisasi: Pilih K titik acak sebagai centroid awal.
2. Assignment: Hitung jarak setiap data ke semua centroid (biasanya Euclidean Distance).

$$d(x, c) = \sqrt{\sum (x_i - c_i)^2}$$

Data masuk ke centroid terdekat.

3. Update: Geser centroid ke posisi rata-rata dari semua data anggotanya.

$$c_{new} = \frac{1}{N} \sum x_i$$

4. Repeat: Ulangi langkah 2-3 hingga posisi centroid stabil (tidak berubah).

3.3 Tensor & High Performance Computing (HPC)

Data dalam *Machine Learning* adalah **Array**.

- **1D Array:** Vector (List angka).
- **2D Array:** Matrix (Tabel baris x kolom).
- **3D Array:** Tensor (Contoh: Gambar RGB - Tinggi x Lebar x 3 Channel).
- **N-D Array:** Tensor dimensi tinggi.

Mengapa Tensor (PyTorch/TensorFlow)?

Library standar seperti numpy berjalan di CPU (pintar tapi core sedikit). Framework Tensor dirancang untuk GPU (ribuan core bodoh tapi bisa kerja bareng/paralel). Ini krusial untuk perkalian matriks raksasa dalam Deep Learning.

4. Implementation Code (PyTorch Tensor & Clustering Visualization)

Berikut adalah contoh kode implementasi operasi Tensor sederhana menggunakan PyTorch (jika tersedia) atau NumPy, serta visualisasi perbandingan *Clustering*.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture

# Tensor / Matrix Operation
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = np.array([[7, 8],
              [9, 1],
              [2, 3]])
C = np.dot(A, B)
print(C)
```

```
# Data Generation
X, _ = make_blobs(n_samples=400, centers=4, cluster_std=0.60, random_state=0)
rng = np.random.RandomState(13)
X = np.dot(X, rng.randn(2, 2))

# K-Means
kmeans = KMeans(n_clusters=4, random_state=42)
labels_kmeans = kmeans.fit_predict(X)

# GMM
gmm = GaussianMixture(n_components=4, random_state=42)
labels_gmm = gmm.fit_predict(X)

# Visualization
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

ax[0].scatter(X[:, 0], X[:, 1], c=labels_kmeans, s=40, cmap='viridis')
ax[0].set_title('K-Means')

ax[1].scatter(X[:, 0], X[:, 1], c=labels_gmm, s=40, cmap='viridis')
ax[1].set_title('GMM')

plt.show()
```

5. Kesimpulan

Dari materi pertemuan ini, dapat disimpulkan:

1. **Fleksibilitas GMM:** GMM lebih unggul daripada K-Means untuk data yang distribusinya tidak bulat sempurna (misal: lonjong), karena menggunakan parameter kovarians (Σ) dan probabilitas, bukan sekadar jarak kaku.
2. **Kekuatan Tensor:** Pemahaman tentang **Tensor** dan operasi matriks adalah fondasi *Deep Learning*. Penggunaan GPU (via PyTorch/TensorFlow) mutlak diperlukan untuk menangani komputasi paralel pada data berdimensi besar.
3. **Dimensionality Reduction:** Teknik seperti **PCA** (Linear) dan **Autoencoder** (Non-linear/Deep Learning) sangat penting untuk memvisualisasikan data berdimensi tinggi ke bentuk 2D/3D agar pola data (*topology*) dapat dipahami oleh manusia.
4. **HPC Awareness:** Mahasiswa diperkenalkan pada konsep *High Performance Computing* (HPC), di mana efisiensi kode dan pemanfaatan *hardware* (CPU vs GPU) menjadi pertimbangan utama dalam membangun sistem AI skala industri.

